Temas Selectos

12 de noviembre de 2026

Moisés Velasco Lozano

PhD

Profesor

Universidad Nacional Autónoma de México

Faculty Research Guest

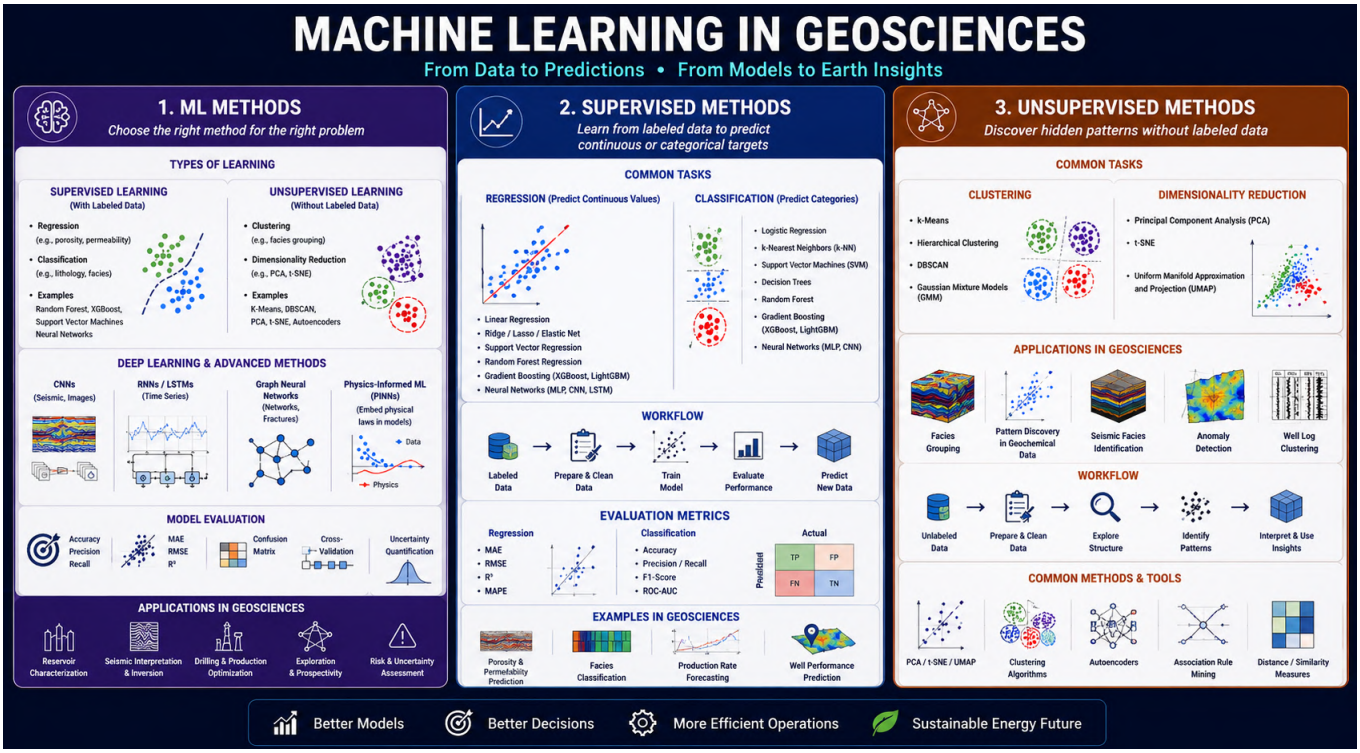
Los Alamos National Laboratory

Índice

1 Machine Learning **4**

1.1 Fundamentos . . . 4
1.2 Entrenamiento del modelo . . . 4
1.2.1 Evaluación de la precisión del modelo (Assessing Model Accuracy) . . . 5
1.2.2 Underfitting . . . 6
1.2.3 Overfitting . . . 6
1.2.4 Curvas de entrenamiento y validación . . . 7
1.2.5 Cross-validation plots . . . 7
1.2.6 Hiperparámetros (Hyperparameters) . . . 9
1.2.7 Ajuste de hiperparámetros (Hyperparameter Tuning) . . . 10
1.3 Métodos de Machine Learning . . . 13
1.4 Métodos de aprendizaje supervisado . . . 13
1.4.1 Linear Regression . . . 14
1.4.2 Ridge Regression . . . 16
1.4.3 Polynomial regression . . . 20
1.4.4 Support Vector Machines . . . 21
1.5 Métodos de aprendizaje no supervisados . . . 26
1.5.1 K-means . . . 28
1.5.2 Density-Based Spatial Clustering of Applications with Noise (DBSCAN) . . . 33
1.5.3 Spectral Clustering . . . 36
1.6 Métodos de aprendizaje semisupervisado . . . 39
1.7 Métodos de aprendizaje por refuerzo . . . 40

4 Machine Learning en Geociencias



1 Machine Learning

La Inteligencia Artificial (IA) es un campo amplio de la computación cuyo objetivo es desarrollar sistemas capaces de realizar tareas que normalmente requieren algún grado de inteligencia humana, como razonar, tomar decisiones, reconocer patrones, comprender lenguaje o resolver problemas.

El Machine Learning (ML) es una rama de la Inteligencia Artificial enfocada en desarrollar algoritmos que pueden aprender patrones y relaciones directamente a partir de los datos, y utilizar ese aprendizaje para realizar predicciones, clasificaciones o identificar estructuras.

El Machine Learning (ML) comprende un conjunto de métodos computacionales capaces de identificar patrones, relaciones y estructuras a partir de los datos, permitiendo desarrollar modelos descriptivos y predictivos sin definir explícitamente todas las relaciones que gobiernan el sistema.

En esta sección se abordarán algunos de los principales métodos de Machine Learning y su aplicación en geociencias y sistemas energéticos del subsuelo, donde existe una gran cantidad de información geológica, petrofísica, geofísica y de producción. Los métodos se agruparán en tres áreas principales: clustering, para identificar grupos o estructuras naturales en los datos; regresión, para predecir variables continuas; y clasificación, para asignar observaciones a diferentes categorías o clases.

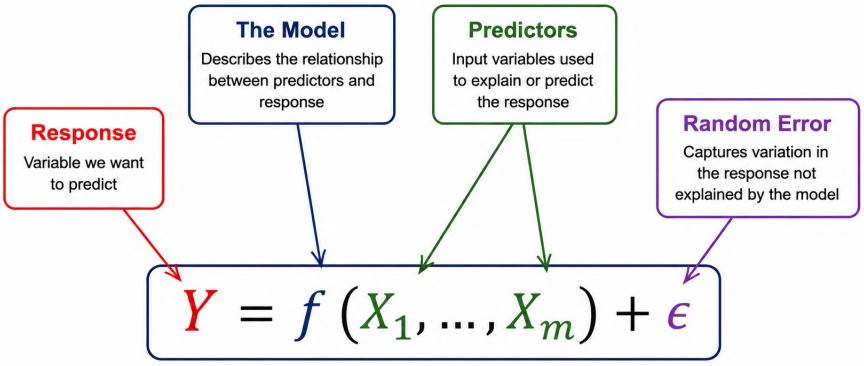
Estas herramientas permiten abordar problemas como la identificación de facies, caracterización de yacimientos, predicción de propiedades petrofísicas, clasificación de litologías y análisis de datos de producción, entre otras aplicaciones.

1.1 Fundamentos

El objetivo de machine learning es estimar el modelo f para realizar una predicción

- **Variables independientes** (predictor features): Información de entrada
- **Variables dependientes** (response features): Información de salida

El objetivo de machine learning es estimar el modelo f para realizar una predicción



1.2 Entrenamiento del modelo

Una vez definidos los conjuntos de entrenamiento, validación y prueba, es necesario determinar si el modelo está aprendiendo correctamente los patrones presentes en los datos y, principalmente, si es capaz de generalizar hacia observaciones que no fueron utilizadas durante el entrenamiento. Un buen desempeño sobre los datos de entrenamiento no garantiza que el modelo produzca predicciones confiables sobre nuevos datos.

La evaluación del entrenamiento debe realizarse de manera continua durante el desarrollo del modelo. Para ello, se comparan las métricas obtenidas en entrenamiento y validación, se analiza la complejidad del modelo, se ajustan

sus hiperparámetros y, cuando es apropiado, se utilizan técnicas como validación cruzada (cross-validation). Este análisis permite identificar dos problemas fundamentales: underfitting y overfitting.

1.2.1 Evaluación de la precisión del modelo (Assessing Model Accuracy)

Una vez entrenado un modelo de machine learning, es necesario determinar qué tan bien reproduce los datos observados y qué tan confiables son sus predicciones. Este proceso se conoce como Assessing Model Accuracy y consiste en utilizar métricas cuantitativas para comparar los valores predichos por el modelo con los valores reales.

En problemas de regresión, donde la variable objetivo es continua (por ejemplo, porosidad, permeabilidad, saturación de agua, presión o producción), la evaluación se basa principalmente en los residuales o errores de predicción:

$$e_i = y_i - \hat{y}_i$$

donde $y_i$representa el valor observado y $\hat{y}_i$ el valor estimado. Un modelo adecuado debería producir errores relativamente pequeños y sin patrones sistemáticos. Una función de pérdida (loss function) cuantifica qué tan diferentes son las predicciones generadas por un modelo respecto a los valores reales. Durante el entrenamiento, el algoritmo busca modificar sus parámetros para minimizar esta función, por lo que la elección de la función de pérdida influye directamente en qué tipo de errores aprende a evitar el modelo. Funciones de pérdida utilizadas en el entrenamiento de modelos de machine learning:

- **Mean Absolute Error (MAE)**

El MAE representa el promedio del valor absoluto de los errores:

$$MAE = \frac{1}{n}\sum |y_i - \hat{y}_i|$$

Es fácil de interpretar porque conserva las mismas unidades de la variable objetivo. Además, es menos sensible a errores extremos que el MSE.

- **Mean Squared Error (MSE)**

El MSE proporciona mayor penalización a predicciones con errores grandes debido al término cuadrático. Es ampliamente utilizado tanto como métrica de evaluación como función de pérdida durante el entrenamiento de modelos de regresión.

$$MSE = \frac{1}{n}\sum (y_i - \hat{y}_i)^2$$

Una desventaja es que sus unidades corresponden al cuadrado de las unidades originales, lo que puede dificultar su interpretación física.

- **Root Mean Squared Error (RMSE)**

El RMSE corresponde a la raíz cuadrada del MSE:

$$RMSE = \sqrt{\frac{1}{n}\sum (y_i - \hat{y}_i)^2}$$

Su principal ventaja es que recupera las unidades originales de la variable objetivo, facilitando la interpretación. Al igual que el MSE, penaliza considerablemente los errores grandes.

COMPARACIÓN DE LAS PRINCIPALES MÉTRICAS DE EVALUACIÓN

Regresión (variable objetivo continua)

MÉTRICA	FÓRMULA	INTERPRETACIÓN	MEJOR VALOR	UNIDADES	CARACTERÍSTICAS PRINCIPALES	SENSIBILIDAD A OUTLIERS	RANGO TÍPICO
MAE (Mean Absolute Error)	$MAE = \frac{1}{n}\sum_{i=1}^{n}\lvert y_i - \hat{y}_i\rvert$	Promedio de la magnitud absoluta de los errores. Mide, en promedio, cuánto se desvían las predicciones de los valores reales.	0 (más cercano a 0, mejor)	Las mismas unidades de la variable objetivo (ej. mD, psi, %)	• Fácil de interpreter • Menos penalización a errores grandes • Robusta y estable	Baja	[0, ∞) Valores más cercanos a 0 son mejores
MSE (Mean Squared Error)	$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$	Promedio de los errores al cuadrado. Penaliza fuertemente los errores grandes.	0 (más cercano a 0, mejor)	Cuadrado de las unidades de la variable objetivo (ej. mD^2, psi^2)	• Muy utilizada como función de pérdida • Penaliza fuertemente errores grandes	Alta	[0, ∞) Valores más cercanos a 0 son mejores
RMSE (Root Mean Squared Error)	$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$	Raíz cuadrada del MSE. Conserva las unidades originales y penaliza errores grandes.	0 (más cercano a 0, mejor)	Las mismas unidades de la variable objetivo (ej. mD, psi, %)	• Fácil de interpretar • Penaliza fuertemente errores grandes • Muy usada en geociencias	Alta	[0, ∞) Valores más cercanos a 0 son mejores
R^2 (Coeficiente de Determinación)	$R^2 = 1 - \frac{\sum_{i=1}(y_i - \hat{y}_i)^2}{\sum_{i=1}(y_i - \bar{y})^2}$	Proporción de la variabilidad de los datos explicada por el modelo. Compara el modelo con usar la media como predicción.	1 (más cercano a 1, mejor)	Adimensional (sin unidades)	• Mide ajuste global • Puede ser negativo si el modelo es peor que usar la media	Media	(−∞, 1] Ideal cercano a 1; puede ser negativo
MAPE (Mean Absolute Percentage Error)	$MAPE = \frac{100}{n}\sum_{i=1}^{n}\left\lvert\frac{y_i - \hat{y}_i}{y_i}\right\rvert$	Error absoluto promedio expresado en porcentaje respecto al valor real.	0 % (más cercano a 0 %, mejor)	Porcentaje (%)	• Intuitivo y fácil de comunicar • Útil para comparar entre diferentes escalas	Alta (problemas si y_i es cercano a 0)	[0, ∞) % Valores más cercanos a 0 % son mejores

NOTA IMPORTANTE: Ninguna métrica debe interpretarse de forma aislada. Es recomendable analizar varias métricas en conjunto y complementarlas con gráficos (observado vs. predicho, residuales) y validación en datos no vistos.

1.2.2 Underfitting

El underfitting ocurre cuando el modelo es demasiado simple para representar adecuadamente las relaciones presentes en los datos. En esta situación, el modelo presenta un desempeño deficiente tanto en los datos de entrenamiento como en los de validación.

Por ejemplo, intentar representar mediante una regresión lineal una relación fuertemente no lineal puede producir underfitting. El modelo no posee suficiente flexibilidad para capturar la estructura de los datos.

Generalmente se observa:

Error de entrenamiento alto + error de validación alto

Este comportamiento se asocia con alto sesgo (high bias). Algunas alternativas para reducirlo incluyen aumentar la complejidad del modelo, incorporar variables relevantes, utilizar transformaciones de las variables, reducir una regularización excesiva o utilizar algoritmos capaces de representar relaciones más complejas.

1.2.3 Overfitting

El overfitting ocurre cuando el modelo aprende excesivamente las particularidades de los datos de entrenamiento, incluyendo ruido, anomalías y patrones que no son representativos del fenómeno general.

En este caso, el modelo puede presentar un error extremadamente bajo durante el entrenamiento, pero un error considerablemente mayor cuando se aplica a los datos de validación o prueba:

Error de entrenamiento muy bajo + error de validación significativamente mayor

El modelo presenta entonces alta varianza (high variance) y una capacidad limitada para generalizar.

El overfitting puede ser particularmente importante cuando se utilizan modelos muy complejos con conjuntos de datos relativamente pequeños. Algunas estrategias para reducirlo incluyen regularización, reducción de la complejidad del modelo, selección de variables, early stopping, dropout en redes neuronales, incorporación de más datos y validación cruzada.

1.2.4 Curvas de entrenamiento y validación

Una herramienta particularmente útil consiste en representar gráficamente la evolución del training loss y validation loss durante el entrenamiento.

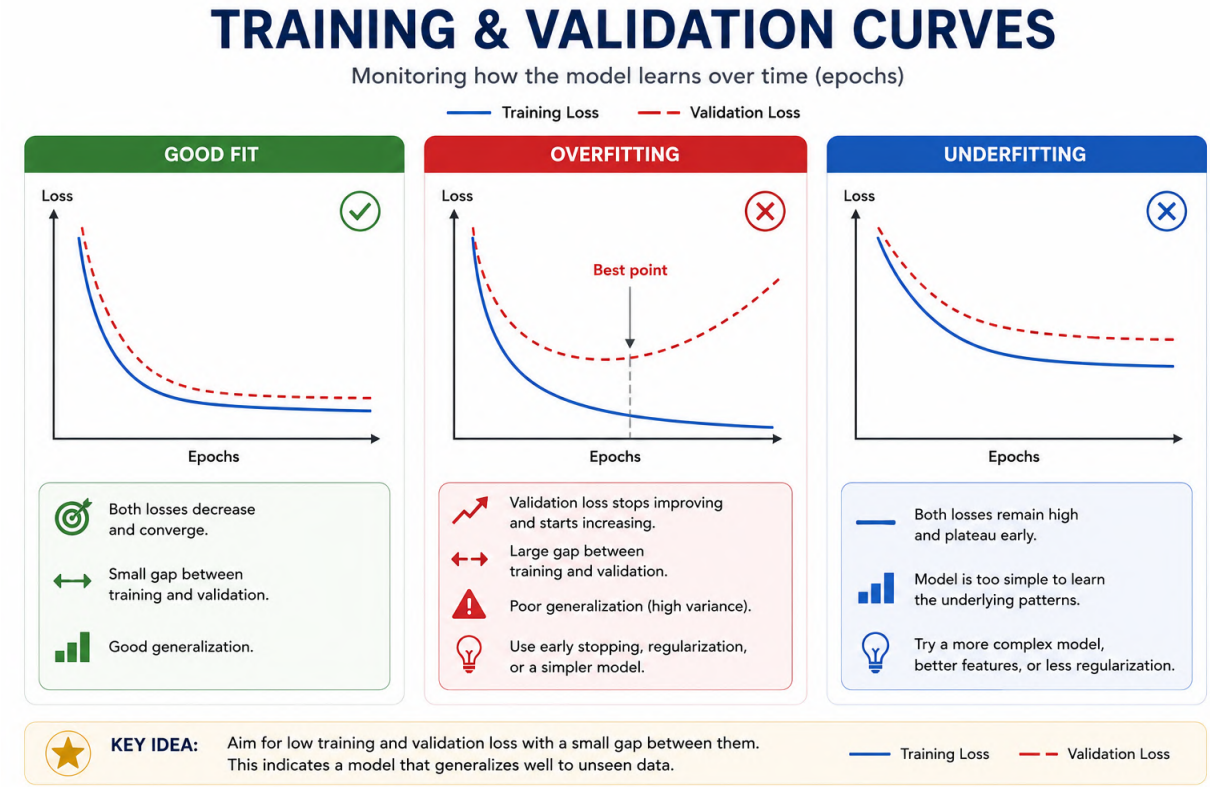
En un entrenamiento adecuado, ambas pérdidas (loss functions) deberían disminuir inicialmente y eventualmente estabilizarse en valores relativamente bajos. No es necesario que sean idénticas; normalmente existe una pequeña diferencia entre ambas.

Una señal clásica de overfitting ocurre cuando:

$$Loss_{train} \downarrow \quad \text{mientras que} \quad Loss_{validation} \uparrow$$

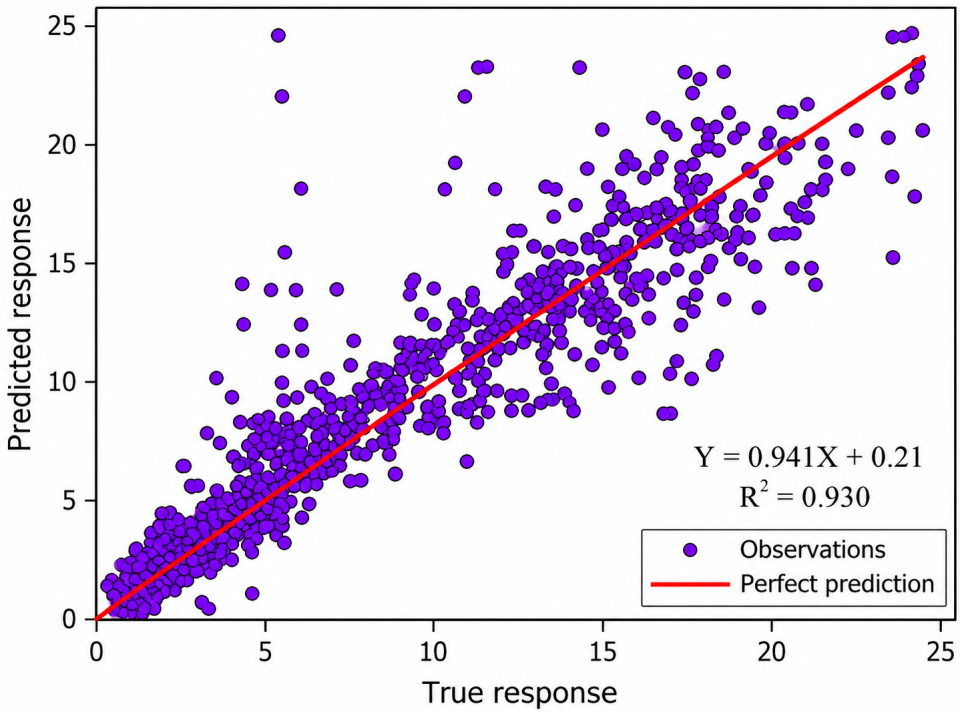
Es decir, el modelo continúa mejorando sobre los datos de entrenamiento mientras comienza a empeorar sobre datos independientes.

En redes neuronales, este comportamiento permite utilizar early stopping: el entrenamiento se detiene cuando la pérdida de validación deja de mejorar durante determinado número de épocas.



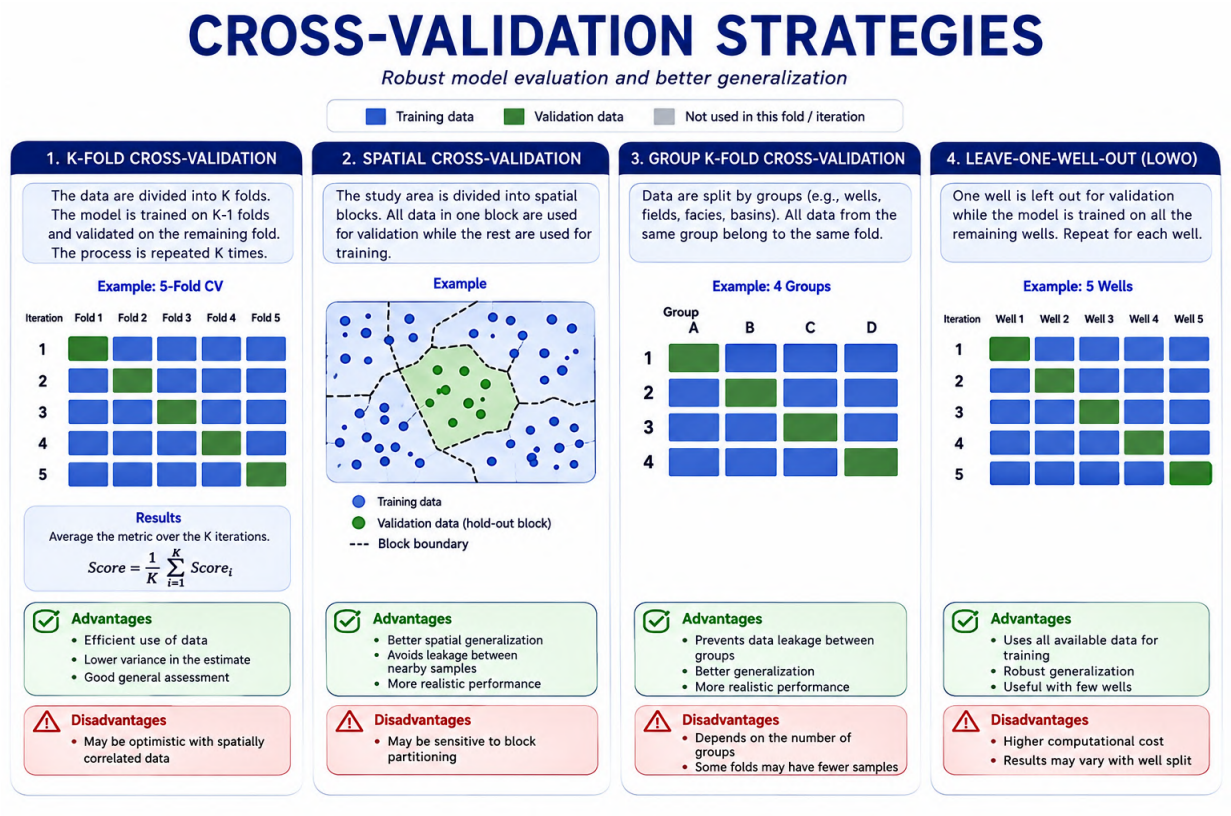
1.2.5 Cross-validation plots

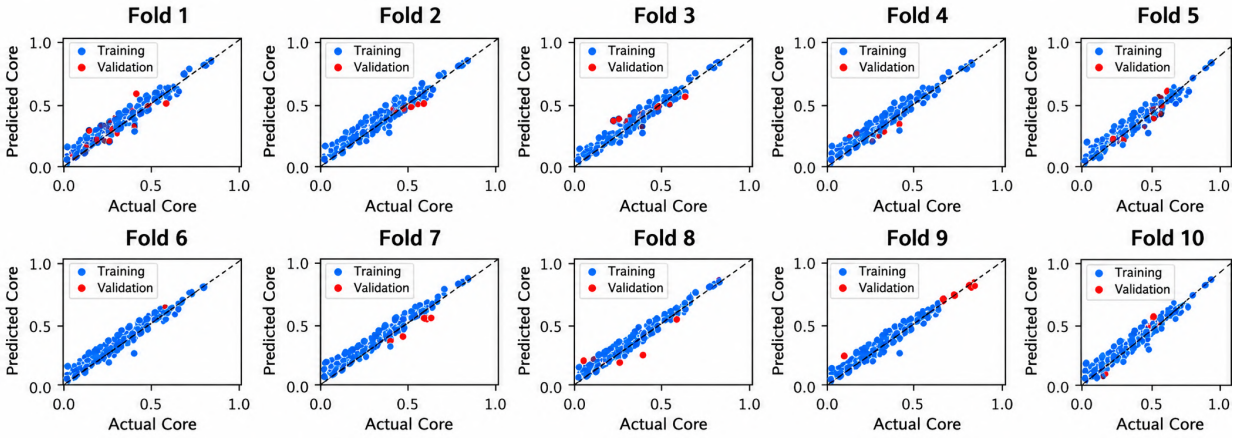
Los cross-validation plots permiten visualizar el comportamiento del modelo entre los diferentes folds y son útiles para evaluar su estabilidad. Pueden mostrar métricas como $R2$, MAE, RMSE, accuracy, precision o recall para cada partición. Un modelo robusto debería presentar valores relativamente consistentes entre los diferentes folds. Una variabilidad elevada puede indicar que el desempeño depende fuertemente de qué observaciones se utilizan para entrenamiento y validación. Otra representación muy útil es la validation curve, donde se analiza cómo cambia el desempeño de entrenamiento y validación al modificar un hiperparámetro que controla la complejidad del modelo.



En aplicaciones de geociencias, un buen resultado de cross-validation debe interpretarse con especial cuidado debido a la autocorrelación espacial. Si muestras espacialmente cercanas aparecen simultáneamente en entrenamiento y validación, las métricas pueden ser artificialmente optimistas.

Por ejemplo, obtener $R2$ =0.95 mediante random K-fold cross-validation no necesariamente significa que el modelo tendrá el mismo desempeño al predecir una región completamente nueva. Cuando el objetivo es realizar predicciones en nuevos pozos, bloques o regiones, resulta recomendable utilizar estrategias como Spatial Cross-Validation, Group K-Fold o Leave-One-Well-Out. De esta manera, la evaluación reproduce mejor las condiciones reales bajo las cuales se utilizará posteriormente el modelo.



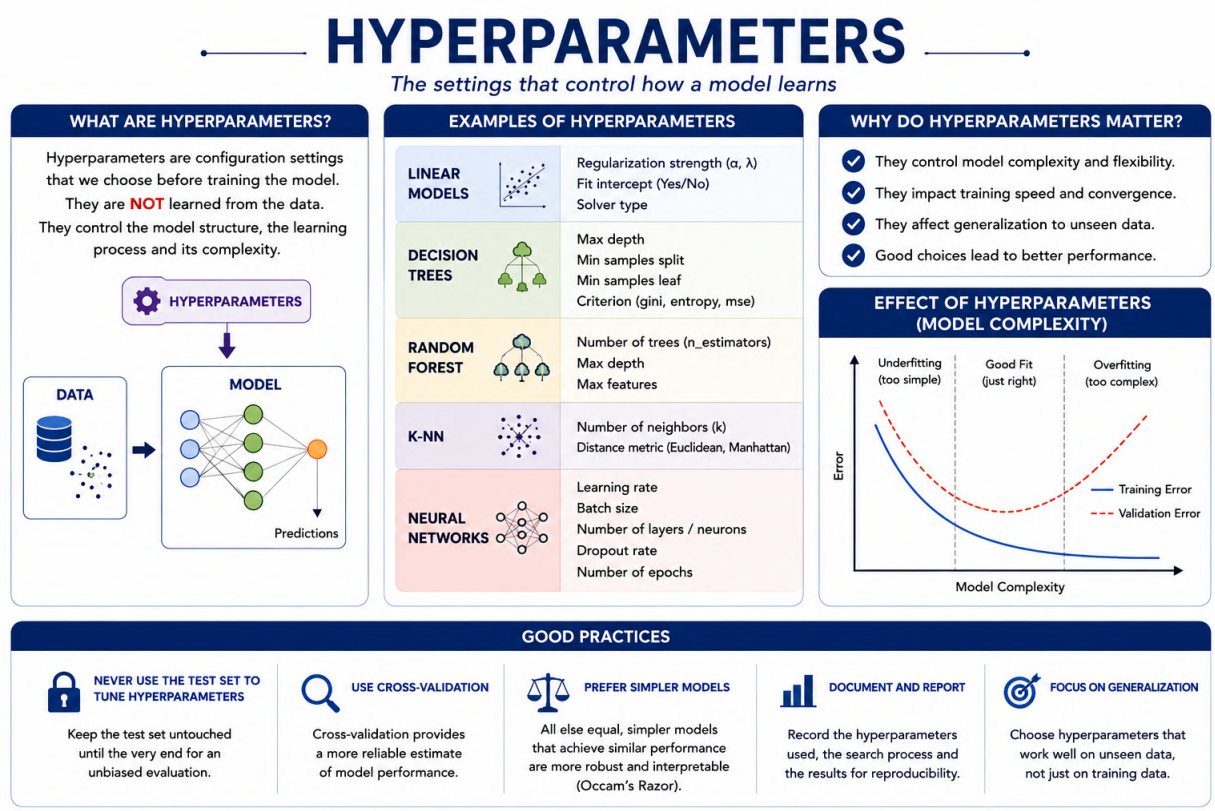


1.2.6 Hiperparámetros (Hyperparameters)

Los hiperparámetros son configuraciones del algoritmo que no son aprendidas directamente a partir de los datos, sino que deben establecerse antes o durante el proceso de entrenamiento. Estos controlan aspectos como la complejidad del modelo, la intensidad de la regularización o la manera en que ocurre el aprendizaje. Los hiperparámetros dependen del algoritmo utilizado. Algunos ejemplos son:

- número máximo de niveles de un árbol de decisión;
- número de árboles en Random Forest;

- número de vecinos k en K-Nearest Neighbors;
- parámetros de regularización en Ridge o Lasso;
- learning rate, batch size y número de épocas en redes neuronales;
- número de capas, neuronas, filtros o tamaño de los kernels en deep learning.



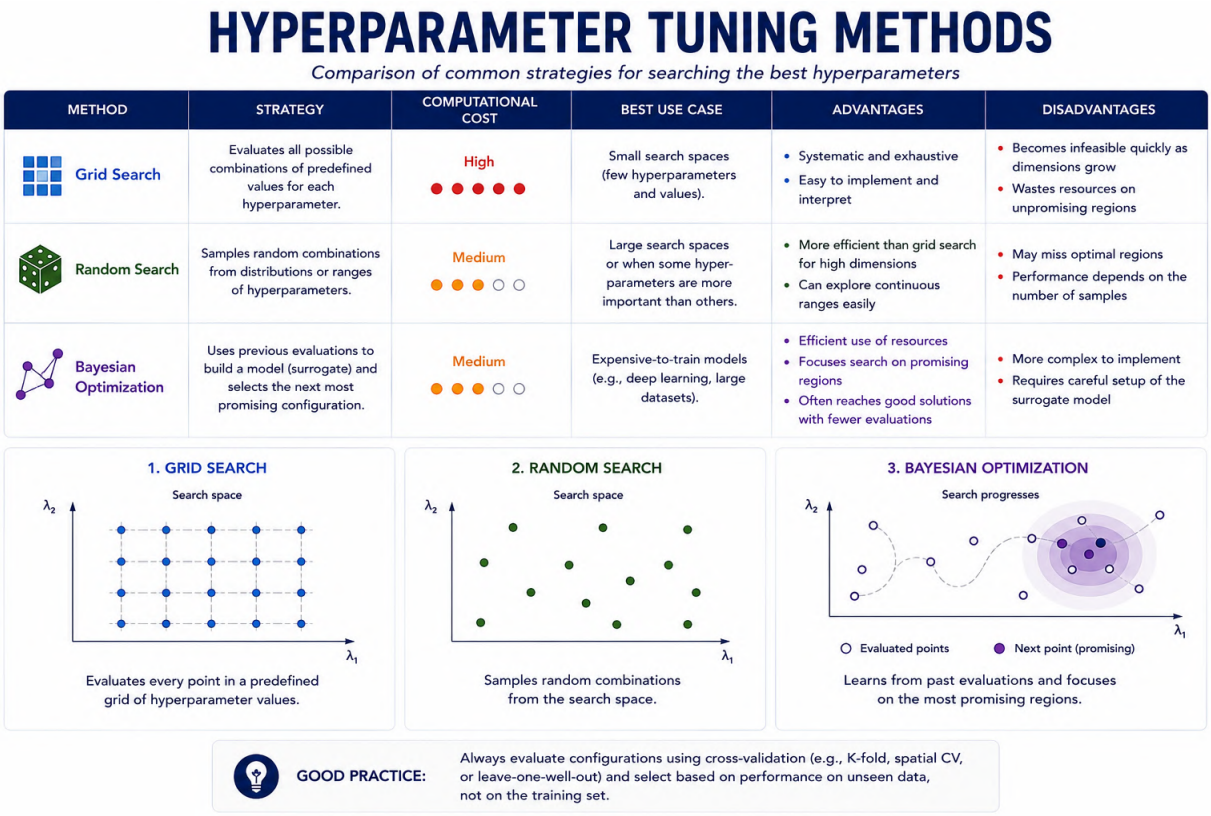
La selección de hiperparámetros debe realizarse utilizando el conjunto de validación o mediante cross-validation, y no con el conjunto de prueba. Algunas estrategias comunes son Grid Search, Random Search y métodos de optimización Bayesiana. El objetivo no es necesariamente encontrar la configuración que produzca el menor error de entrenamiento, sino aquella que presente el mejor desempeño sobre datos no utilizados para ajustar los parámetros del modelo.

1.2.7 Ajuste de hiperparámetros (Hyperparameter Tuning)

Una vez seleccionado un algoritmo, es necesario determinar la configuración de hiperparámetros que permita obtener el mejor desempeño posible. Este proceso se conoce como hyperparameter tuning y consiste en explorar diferentes combinaciones de valores para identificar aquellas que permiten al modelo generalizar adecuadamente a datos no observados. A diferencia de los parámetros internos del modelo, que se estiman durante el entrenamiento, los hiperparámetros son definidos externamente. Por ejemplo, en un Random Forest pueden ajustarse el número de árboles y su profundidad máxima; mientras que en una red neuronal pueden modificarse el learning rate, batch size, número de capas, número de neuronas y regularización.

El objetivo del hyperparameter tuning no es minimizar el error de entrenamiento, sino encontrar la configuración que produzca el mejor desempeño de validación. Para ello, las diferentes configuraciones deben compararse utilizando un conjunto de validación o, preferentemente, cross-validation. El conjunto de prueba debe mantenerse independiente durante todo este proceso. Existen diferentes estrategias para realizar esta búsqueda.

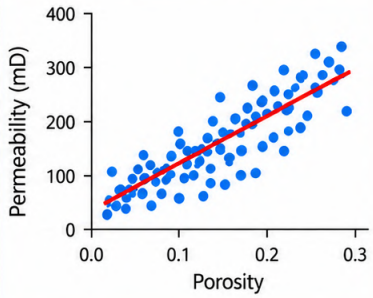
- **Grid Search** es uno de los métodos más sencillos. Consiste en definir previamente un conjunto de valores posibles para cada hiperparámetro y evaluar todas las combinaciones posibles.
- **Random Search** selecciona aleatoriamente combinaciones de hiperparámetros dentro de rangos o distribuciones previamente definidos. En lugar de evaluar todas las combinaciones posibles, se establece un número determinado de configuraciones que serán exploradas.
- **La optimización Bayesiana (Bayesian Optimization)** utiliza los resultados obtenidos en evaluaciones anteriores para decidir qué combinación de hiperparámetros debería evaluarse posteriormente. A diferencia de Grid Search y Random Search, donde cada configuración se selecciona sin utilizar directamente la información de las evaluaciones anteriores, la optimización Bayesiana construye un modelo aproximado de la relación hiperparametros-validación.



Model Parameters

Values **learned from the training data** during the fitting process. You do **not** set them manually.

Example: Linear regression (porosity → permeability)



$$\hat{k} = \beta_0 + \beta_1 \phi$$

Intercept (β_0) — **Parameter** (learned from data)

Slope (β_1) — **Parameter** (learned from data)

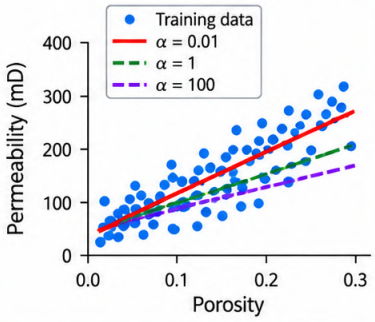
Learned parameters after training:

- β_0 (intercept)
- β_1 (slope/coefficient)

Model Hyperparameters

Configuration choices that **you set before training** (or select using validation). They are **not** learned directly from the training data.

Example: Ridge regression (hyperparameter α)



$$\hat{k} = \beta_0 + \beta_1 \phi$$

+ regularization controlled by α

Hyperparameter (α)

Chosen by the user or using validation

Hyperparameter:

- α (regularization strength)
  (set before training or selected using validation)

1.3 Métodos de Machine Learning

Los métodos de Machine Learning (ML) pueden clasificarse de acuerdo con la forma en que los algoritmos utilizan los datos durante el proceso de aprendizaje. La principal diferencia radica en si los datos contienen una variable objetivo conocida (label o target), si el algoritmo debe descubrir estructuras por sí mismo o si aprende mediante la interacción con un entorno.

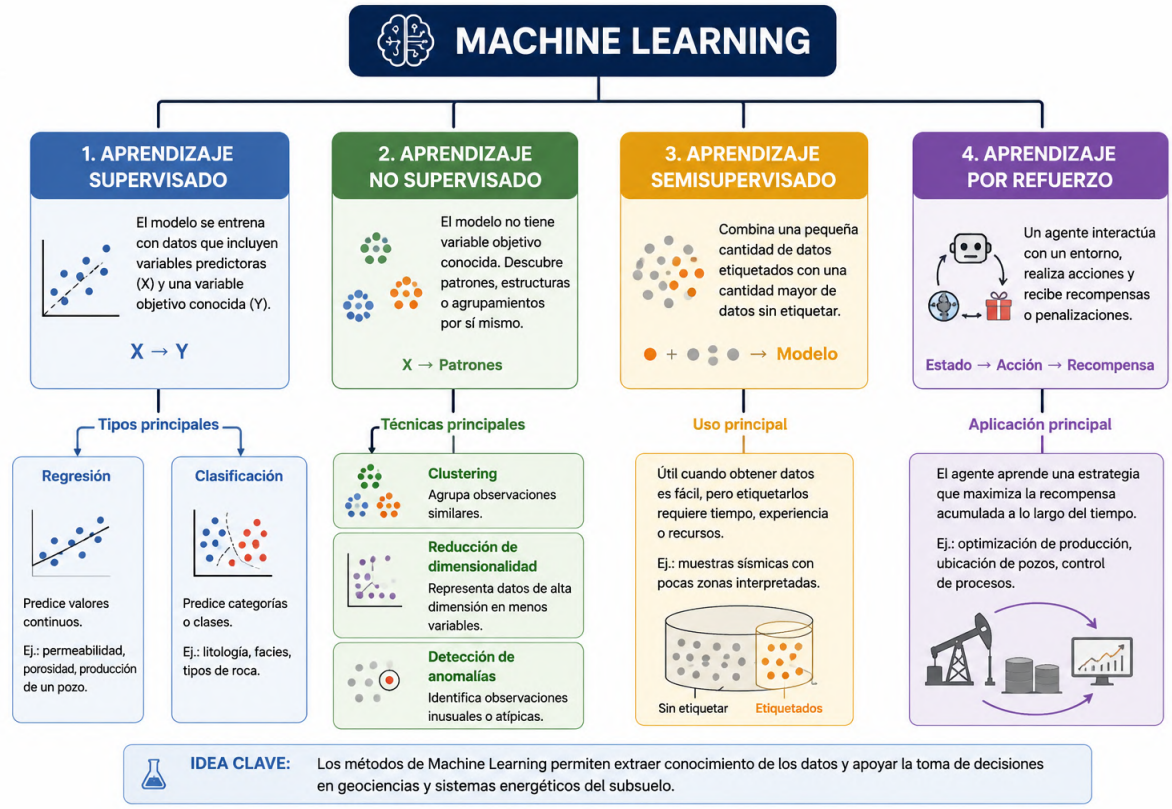


Figura 1: Métodos de Machine Learning

1.4 Métodos de aprendizaje supervisado

En el aprendizaje supervisado, el modelo se entrena utilizando ejemplos para los cuales se conoce tanto la información de entrada como la respuesta correcta. El objetivo es aprender una relación entre las variables predictoras X y la variable objetivo Y, para posteriormente realizar predicciones sobre nuevos datos.

Durante el entrenamiento, el modelo compara sus predicciones con los valores reales y ajusta sus parámetros para reducir el error de predicción.

$$X \longrightarrow Y$$

Los problemas supervisados se dividen principalmente en:

1. **Regresión**: cuando Y es una variable numérica continua. Por ejemplo, predecir porosidad, permeabilidad, presión o producción.
2. **Clasificación**: cuando Y representa una categoría o clase. Por ejemplo, identificar facies, litologías o tipos de roca.

Entre los algoritmos utilizados se encuentran regresión lineal y logística, KNN, árboles de decisión, Random Forest, Support Vector Machines (SVM), Gradient Boosting y redes neuronales.

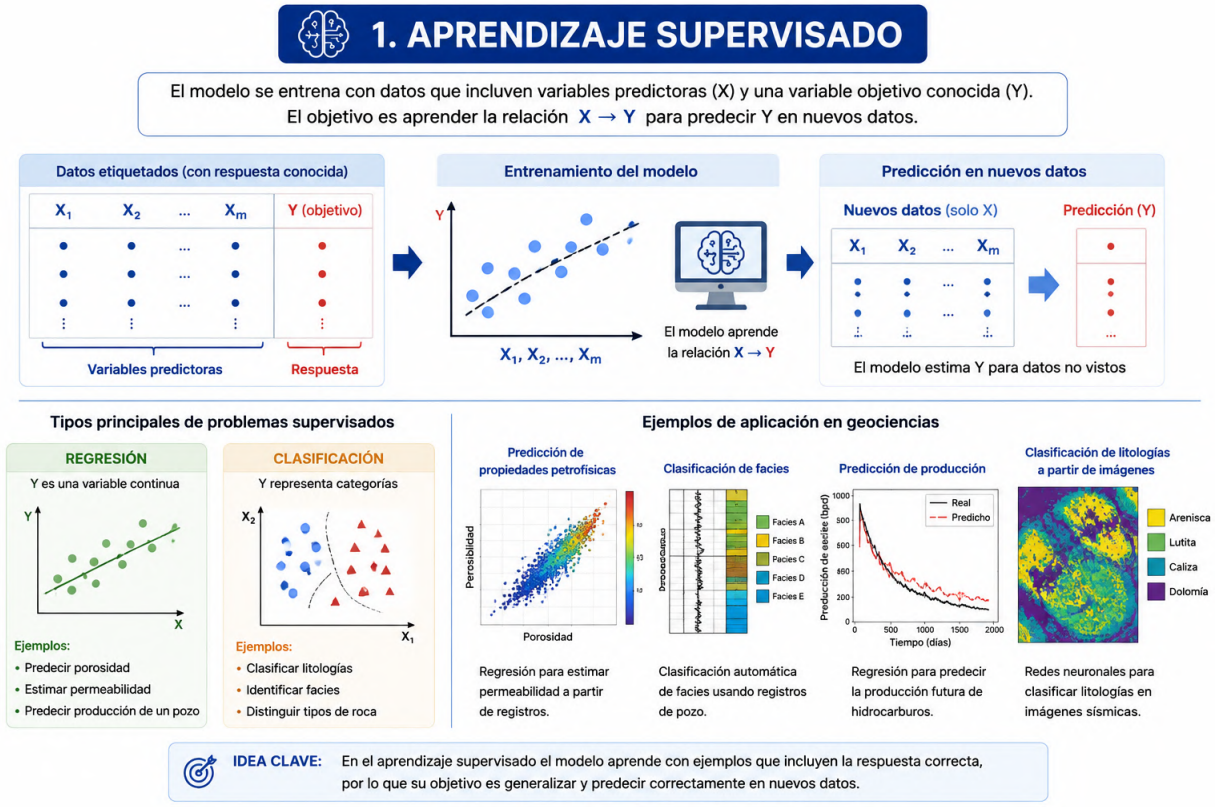


Figura 2: Supervised machine learning

Idea clave: el modelo aprende utilizando ejemplos que incluyen la respuesta que se desea predecir.

1.4.1 Linear Regression

La regresión lineal es uno de los métodos fundamentales del aprendizaje supervisado y constituye una excelente introducción a los modelos predictivos de Machine Learning. Su objetivo es modelar la relación entre una variable objetivo continua Y y una o más variables predictoras X mediante una relación lineal.

A partir de observaciones donde conocemos tanto las variables de entrada como la respuesta, el modelo aprende los parámetros que mejor describen esta relación. Una vez entrenado, puede utilizarse para estimar el valor de Y para nuevas observaciones.

Por ejemplo, en geociencias podemos utilizar la porosidad como variable predictora x para estimar la permeabilidad y.

Cuando se utiliza una sola variable predictora, el modelo puede representarse como:

$$\hat{y} = xb_1 + b_0$$

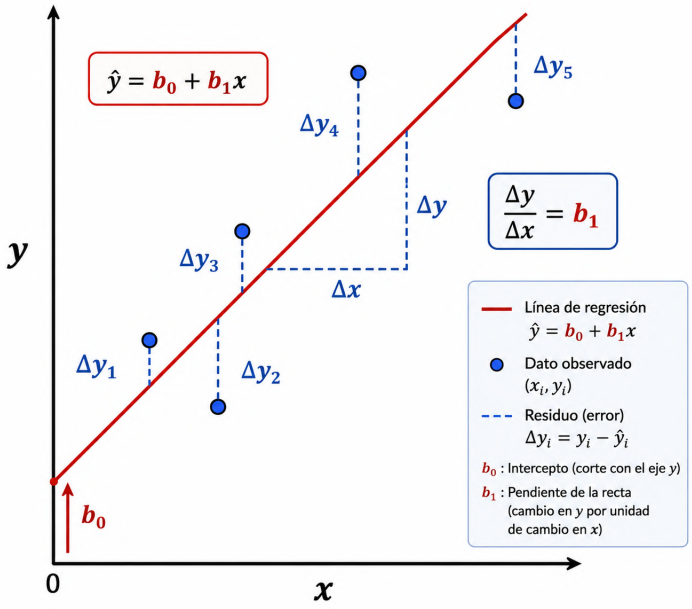
donde:
x: variable predictora o feature.
y: valor real de la variable objetivo.
$\hat{y}$: valor predicho por el modelo.
b_0: intercepto de la recta.
b_1: pendiente o coeficiente de regresión.

El coeficiente $b_1$representa cuánto se espera que cambie y por cada unidad de incremento en x. Por ejemplo, si $b_1 > 0$, la respuesta aumenta conforme aumenta el predictor.

Sin embargo, en los modelos se tiene una diferencia entre los valores reales y los valores predecidos, es decir se tiene un residuo:

$$\Delta y_i = y_i - \hat{y}_i$$

Los residuos permiten cuantificar los errores que comete el modelo. Cuanto menores sean, mejor será el ajuste de la línea a los datos, aunque un buen ajuste en entrenamiento no garantiza por sí solo una buena generalización.



El método más común es Mínimos Cuadrados Ordinarios (Ordinary Least Squares, OLS). El objetivo es encontrar b_1 y b_o que minimicen la suma de los errores al cuadrado:

$$OLS = \sum (y_i - \hat{y}_i)^2$$

El objetivo es minimizar esta diferencia pero para la totalidad de los valores en el modelo:

$$\sum (\Delta y_i)^2 = \sum [y_i - (x_i b_1 + b_0)]^2$$

En general esto se denomina la función de pérdida (**loss function**).

En el caso que se tengan múltiples predictors, el modelo se expresa:

$$\hat{y} = b_0 + b_1 X_1 + b_2 X_2 + \cdots + b_m X_m$$

donde la función de pérdida es:

$$\sum (\Delta y_i)^2 = \sum_i [y_i - (\sum_j x_{i,j} b_j + b_0)]^2$$

Ejemplo en geociencias. Supongamos que queremos predecir la producción:

$$Q = f(\phi, k, h, p, Sw, Vsh, TOC, \ldots)$$

Sin embargo, existe una limitación con la aplicación de la regresión lineal ya que algunas de estas propiedades pueden estar correlacionadas. Por ejemplo, ciertos registros de pozo pueden proporcionar información parcialmente redundante sobre la litología o la porosidad.

Una regresión lineal convencional podría asignar coeficientes muy grandes y opuestos a variables correlacionadas. Ridge estabiliza estos coeficientes, distribuyendo de manera más controlada su influencia sobre la predicción.

1.4.2 Ridge Regression

La regresión Ridge es una extensión de la regresión lineal que incorpora un término de regularización para controlar la magnitud de los coeficientes del modelo. Su principal objetivo es obtener modelos más estables y con mejor capacidad de generalización, especialmente cuando existen muchas variables predictoras o cuando algunas de ellas están fuertemente correlacionadas.

En una regresión lineal múltiple tenemos:

$$\hat{y} = \sum b_j X_j + b_0$$

$$\hat{y} = b_0 + b_1 X_1 + b_2 X_2 + \cdots + b_m X_m$$

Ridge agrega una penalización sobre el tamaño de los coeficientes:

$$Loss_{Ridge} = \sum (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1} b_j^2$$

Generalmente el intercepto b_0 no se penaliza por lo que se tiene:

$$Loss_{Ridge} = \sum_i [y_i - (\sum_j x_{i,j} b_j + b_0)]^2 + \lambda \sum_{j=1} b_j^2$$

El segundo término se conoce como **penalización L2**. Aquí:

- b_j son los coeficientes asociados con las variables predictoras.
- λ es el parámetro de regularización.
- Valores grandes de b_j producen una penalización mayor.
- Normalmente, el intercepto b_0 no se penaliza.

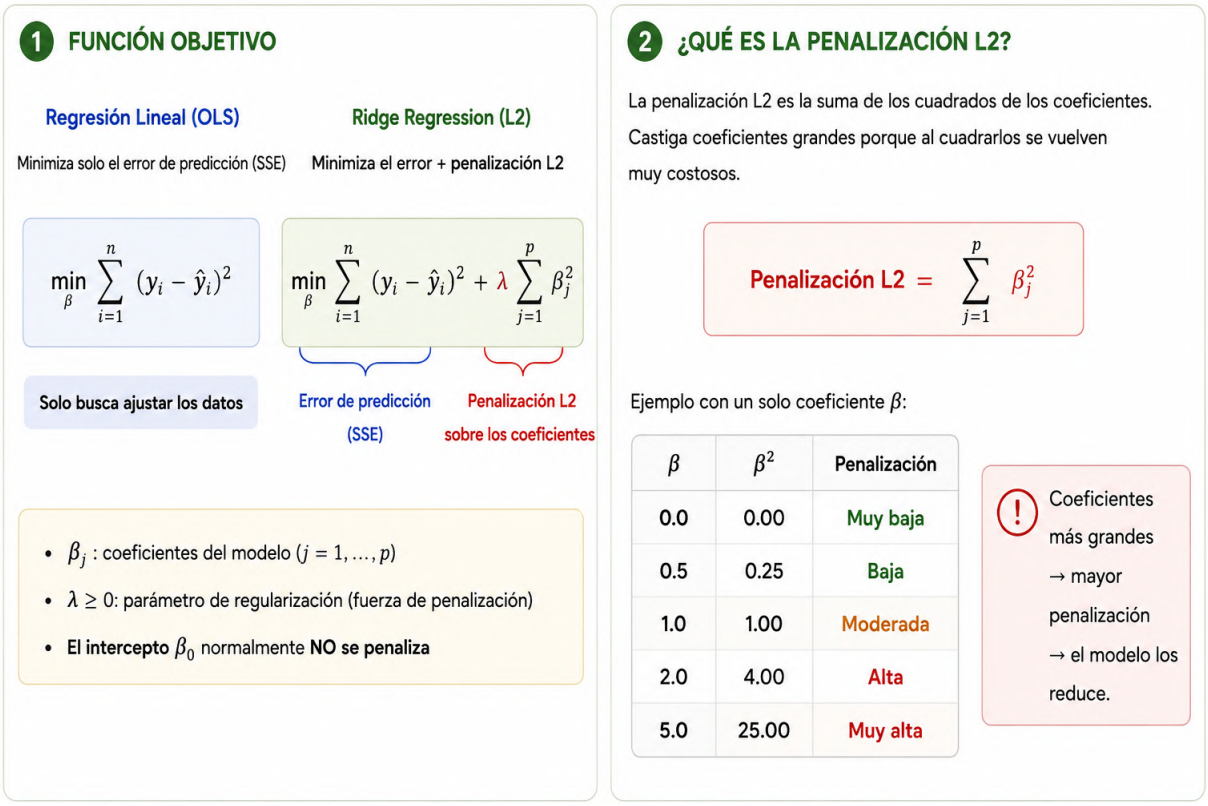


Figura 3: Linear regression vs Ridge regression

El parámetro λ controla el equilibrio entre ajustarse a los datos de entrenamiento y mantener pequeños los coeficientes.

Si:

$$\lambda = 0$$

no existe penalización y Ridge se reduce esencialmente a una regresión lineal ordinaria.
Si:

$$\lambda \uparrow$$

aumenta la penalización y los coeficientes son progresivamente reducidos hacia cero:

$$| b_j | \downarrow$$

Sin embargo, una característica importante de Ridge es que generalmente reduce los coeficientes sin llevarlos exactamente a cero.

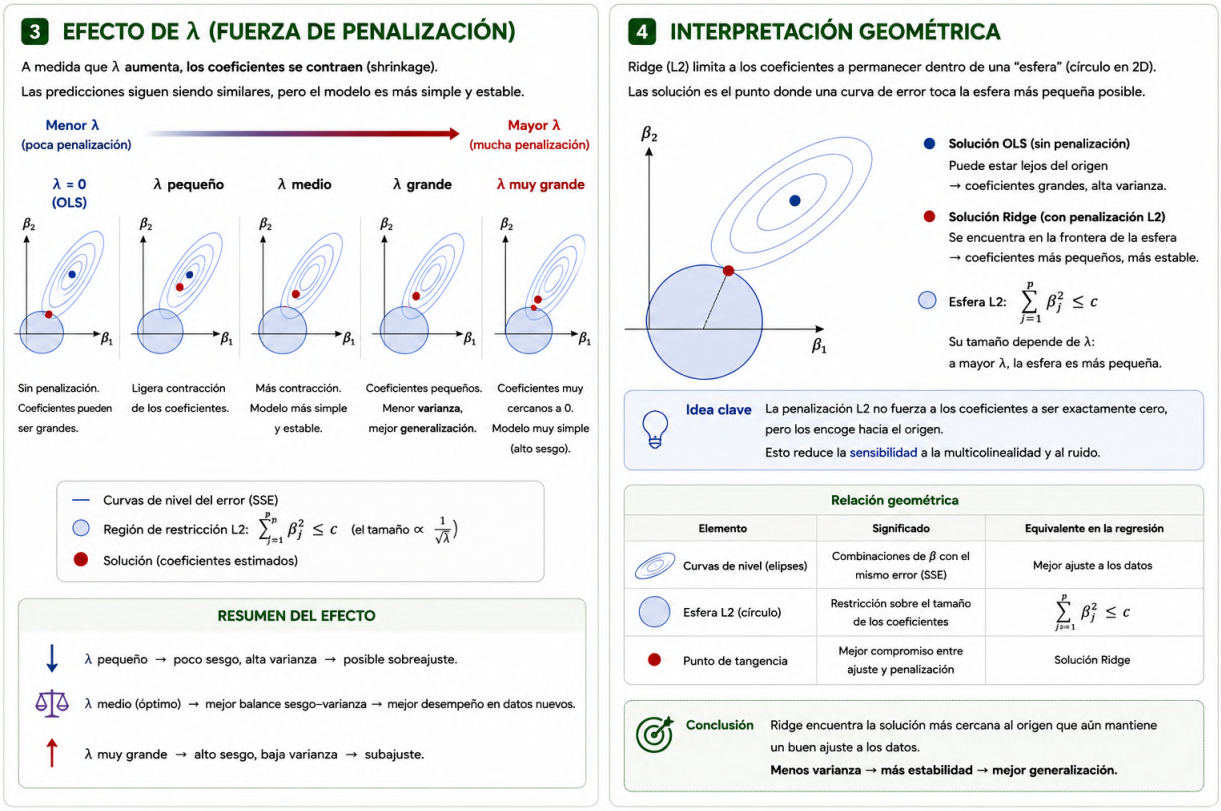


Figura 4: Regularization in Ridge regression

¿Por qué reducir los coeficientes?
Cuando un modelo tiene muchas variables o existen predictores altamente correlacionados, la regresión lineal puede producir coeficientes muy grandes e inestables. Pequeños cambios en los datos pueden entonces generar cambios importantes en el modelo.

Ridge introduce deliberadamente un poco de sesgo (bias) a cambio de reducir la varianza (variance) del modelo. Este compromiso se conoce como **Bias–Variance Tradeoff** y puede mejorar el desempeño sobre datos nuevos.

Variables correlacionadas
Supongamos que queremos predecir la producción de un pozo q utilizando diferentes propiedades del yacimiento:

$$q = b_0 + b_1\phi + b_2k + b_3h + b_4p + b_5S_w + b_6V_{sh} + b_7TOC + \epsilon$$

En un problema real de geociencias, estas variables no necesariamente proporcionan información completamente independiente. Algunas pueden estar fuertemente relacionadas entre sí.

Por ejemplo, podríamos tener dos registros de pozo:

$$X_1 = \text{registro de densidad}$$

$$X_2 = \text{registro neutrón}$$

Ambos contienen información relacionada con la porosidad de la formación. Por lo tanto, pueden presentar una correlación considerable:

$$X_1 \longleftrightarrow X_2$$

El problema es que el modelo recibe dos variables que explican parcialmente la misma información.

Regresión lineal

Supongamos que el modelo obtiene:

$$q = 100 + 25X_1 - 23X_2 + \cdots$$

Los coeficientes $+25$ y -23 son relativamente grandes y de signo contrario.

¿Por qué puede suceder esto?

Como X_1 y X_2contienen información similar, existen diferentes combinaciones de sus coeficientes capaces de producir predicciones parecidas. El modelo puede compensar un coeficiente con el otro:

$$25X_1 - 23X_2$$

Si las dos variables presentan valores similares, gran parte de estos términos puede cancelarse.

Esto puede producir un modelo que predice razonablemente bien los datos de entrenamiento, pero cuyos coeficientes son muy sensibles a pequeñas modificaciones en los datos.

Por ejemplo, utilizando otro conjunto de muestras podríamos obtener:

$$q = 100 + 18X_1 - 16X_2 + \cdots$$

o incluso:

$$q = 100 - 10X_1 + 12X_2 + \cdots$$

Aunque las predicciones globales podrían seguir siendo similares, la interpretación de los coeficientes cambia considerablemente. Este problema está relacionado con la **multicolinealidad**.

Ridge regression

Ridge modifica el problema de optimización agregando una penalización a coeficientes grandes:

$$Loss = OLS + \lambda(b_1^2 + b_2^2 + \cdots + b_m^2)$$

Ahora el modelo no solamente busca:

"¿Qué coeficientes producen el menor error?"

También considera:

"¿Puedo obtener un error similar utilizando coeficientes más pequeños?"

Por ejemplo, en lugar de:

$$25X_1 - 23X_2$$

Ridge podría favorecer una solución más estable como:

$$4.2X_1 + 3.8X_2$$

Ridge tiende a reducir la magnitud de los coeficientes y, ante predictores correlacionados, suele repartir la influencia entre ellos de forma más estable. Sin embargo, no garantiza que todos adopten el mismo signo. La Figura muestra una comparación de los coeficientes con cada unos los esquemas y la elección del hyperparametro λ.

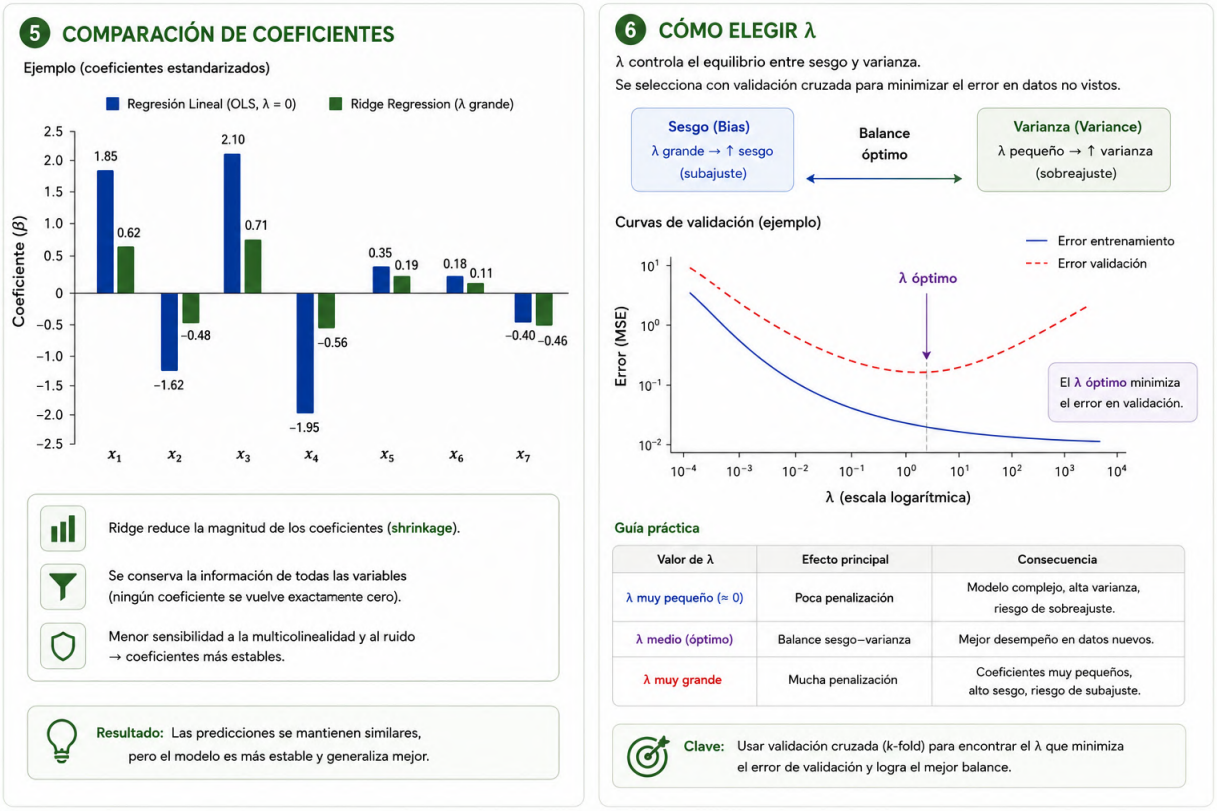


Figura 5: Tuning of hyperparameter λ

Comparación de los métodos de regresión lineal y regresión Ridge

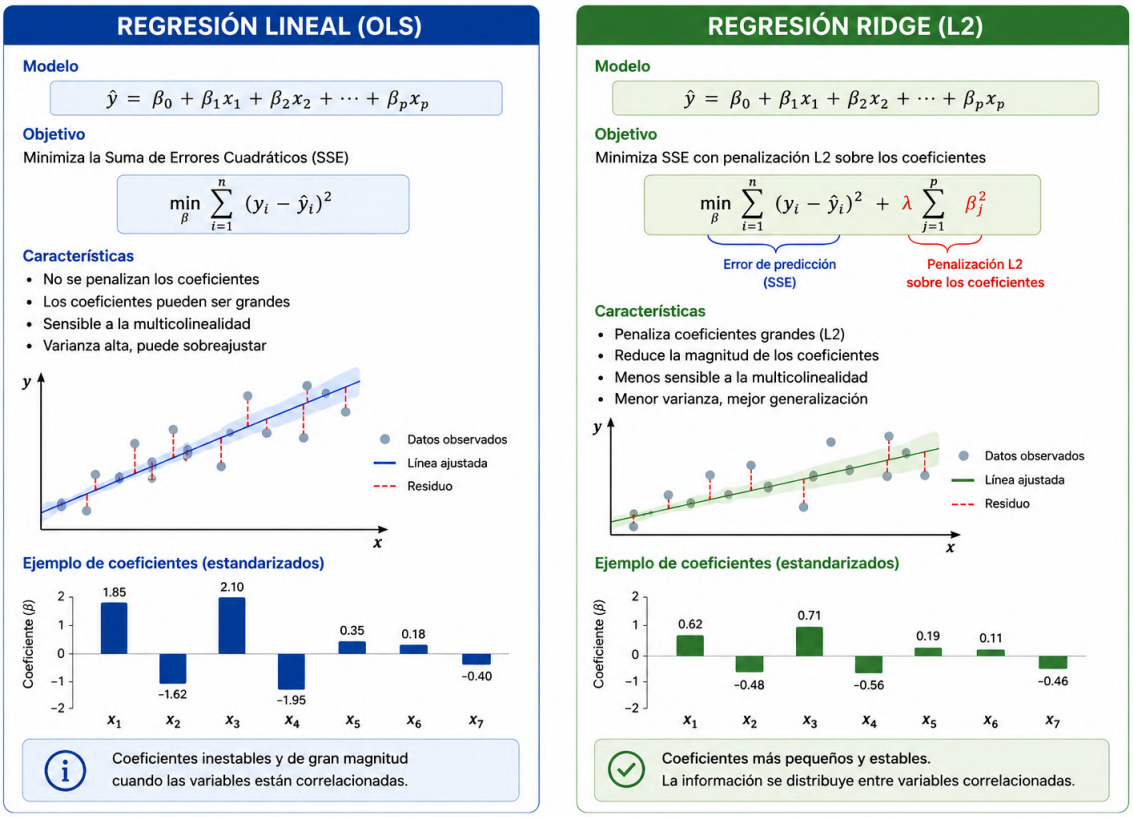


Figura 6: Comparison of results of Linear regression and Ridge regression

1.4.3 Polynomial regression

Para nuestro modelo de regresión lineal tenemos:

$$\hat{y} = \sum b_j X_j + b_0$$

Para el caso de regresión polinomial tenemos:

$$\hat{y} = \sum_l \sum_j b_{j,l} h_l(X_j) + b_0$$

donde:

$$h_1 = X$$

$$h_2 = X^2$$

$$h_3 = X^3$$

Ejemplo: Orden 2 ($l = 2$) y 1 feature ($j = 1$)

$$\hat{y} = b_{1,1} h_1(X_1) + b_{1,2} h_2(X_1) + b_0$$

por ejemplo para orden 3 ($l = 3$) y 3 features ($j = 3$) tenemos:

$$\hat{y} = b_o + b_{1,1}h_1(X_1) + b_{2,1}h_1(X_2) + b_{3,1}h_1(X_3) + b_{1,2}h_2(X_1) + b_{2,2}h_2(X_2) + b_{3,2}h_2(X_3) + b_{1,3}h_3(X_1) + b_{2,3}h_3(X_2) + b_{3,3}h_3(X_3)$$

en términos simples:

$$\hat{y} = b_0 + b_{1,1}X_{1,1} + b_{2,1}X_{1,2} + b_{3,1}X_{1,3} + b_{1,2}X_{2,1}^2 + b_{2,2}X_{2,2}^2 + b_{3,2}X_{3,2}^2 + b_{1,3}X_{3,1}^3 + b_{2,3}X_{3,2}^3 + b_{3,3}X_{3,3}^3$$

en forma reducida se tiene:

$$\hat{y} = \sum_{l,j} b_{j,l} h_l(X_j) + b_0$$

1.4.4 Support Vector Machines

Las Máquinas de Vectores de Soporte (SVM) son algoritmos de aprendizaje supervisado utilizados principalmente para problemas de clasificación, aunque también pueden emplearse para regresión mediante una variante denominada Support Vector Regression (SVR).

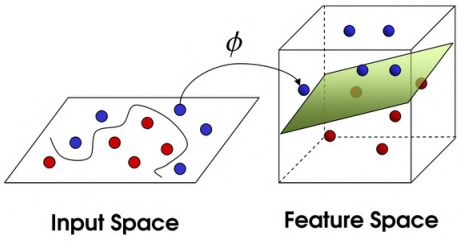


Figura 7: Projection in a higher dimension to separate data with SVM

La idea fundamental de SVM consiste en encontrar una frontera de decisión que permita separar observaciones pertenecientes a diferentes clases. A diferencia de otros métodos que simplemente buscan una frontera capaz de separar los datos, SVM busca aquella que produzca el mayor margen posible entre las clases.

Por ejemplo, en geociencias podríamos utilizar porosidad y permeabilidad para clasificar muestras de roca:

$$X = [\phi, k]$$

$$y = [FaciesA,\ FaciesB]$$

El objetivo de SVM sería encontrar la frontera que mejor separe ambas facies. La geometría depende del número de variables:

- 1 variable: la frontera es un punto.
- 2 variables: la frontera es una línea.
- 3 variables: la frontera es un plano.
- Más de 3 variables: se denomina hiperplano.

Por ejemplo, con porosidad y permeabilidad:

$$[\phi, k] \rightarrow \text{línea de separación}$$

mientras que con porosidad, permeabilidad y Gamma Ray:

$$[\phi, k, GR] \rightarrow \text{plano de separación}$$

En el método se deben entender unos conceptos claves:

- Hiperplano: Frontera de decisión que separa diferentes clases en el espacio de características. En una clasificación lineal, se representa mediante la ecuación $w^T x + b = 0$
- Vectores de soporte (Support Vectors): Puntos de datos más cercanos al hiperplano, fundamentales para determinar la posición del hiperplano y el margen de separación en SVM.
- Margen (Margin): Distancia entre el hiperplano y los vectores de soporte. El objetivo de SVM es maximizar este margen para mejorar la capacidad de generalización del modelo.
- Margen rígido (Hard Margin): Hiperplano de máximo margen que separa perfectamente las diferentes clases de datos, sin permitir errores de clasificación.
- Margen flexible (Soft Margin): Permite ciertos errores de clasificación mediante la introducción de variables de holgura (slack variables), estableciendo un equilibrio entre la maximización del margen de separación y la penalización de los errores cuando los datos no son perfectamente separables.

- Kernel (Función núcleo): Función que permite calcular productos internos en un espacio de características de mayor dimensión, facilitando que SVM clasifique datos que no son linealmente separables en el espacio original.
- C (Parámetro de regularización): Hiperparámetro que controla el equilibrio entre la maximización del margen de separación y la penalización de los errores de clasificación. Un valor elevado de C impone penalizaciones más estrictas a los errores, mientras que un valor pequeño permite mayor tolerancia a estos, favoreciendo un margen de separación más amplio.

Hiperplano de separación. En el caso más sencillo, cuando los datos son linealmente separables, SVM construye un hiperplano definido por:

$$w^T x + b = 0$$

por ejemplo para el caso de tener dos precictors como porosidad y permeabilidad:

$$w = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$$

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

entonces:

$$w^T x = \begin{bmatrix} w_1 & w_2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$w^T x = w_1 x_1 + w_2 x_2$$

donde:
- w (Vector normal): Vector perpendicular al hiperplano que determina su orientación en el espacio de características.
- b (Sesgo o término independiente): Parámetro que determina el desplazamiento del hiperplano respecto al origen a lo largo de la dirección del vector normal w. La distancia perpendicular del hiperplano al origen está dada por $\frac{|b|}{\|w\|}$ y la distancia entre un punto x_i y el hiperplano es $d_i = \frac{w^T x_i + b}{||w||}$

Clasificación lineal de SVM:

$$y = \begin{cases} 1 & w^T x + b \geq 0 \\ -1 & w^T x + b < 0 \end{cases}$$

Objetivo:
Para un conjunto de datos linealmente separable, el objetivo es encontrar el hiperplano que maximice el margen de separación entre las dos clases, garantizando que todas las observaciones sean clasificadas correctamente. Esto conduce a la siguiente formulación:

$$\underset{w,b}{min} \ \frac{1}{2} \|w\|^2$$

sujeto a

$$y_i(w^T x_i + b) \geq 1 \ \ for \ \ i = 1, 2, 3, \cdots, m$$

donde:
y_i es la etiqueta de clase (+1 o -1) correspondiente a cada observación del conjunto de entrenamiento
x_i es el vector de características de la $i - ésima$ observación del conjunto de entrenamiento.
m es el número total de observaciones del conjunto de entrenamiento.
The condition $y_i(w^T x_i + b) \geq 1$ ensures that each data point is correctly classified and lies outside the margin.

En presencia de valores atípicos (outliers) o datos que no son linealmente separables, SVM permite ciertos errores de clasificación mediante la introducción de variables de holgura (ζ_i). En consecuencia, el problema de optimización se modifica de la siguiente manera:

$$\underset{w,b}{min} \ \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{m}\epsilon_i$$

sujeto a

$$y_i(w^T x_i + b) \geq 1 - \epsilon_i \ \ y \ \ \epsilon_i \geq 0 \ \ for \ \ i = 1, 2, 3, \cdots, m$$

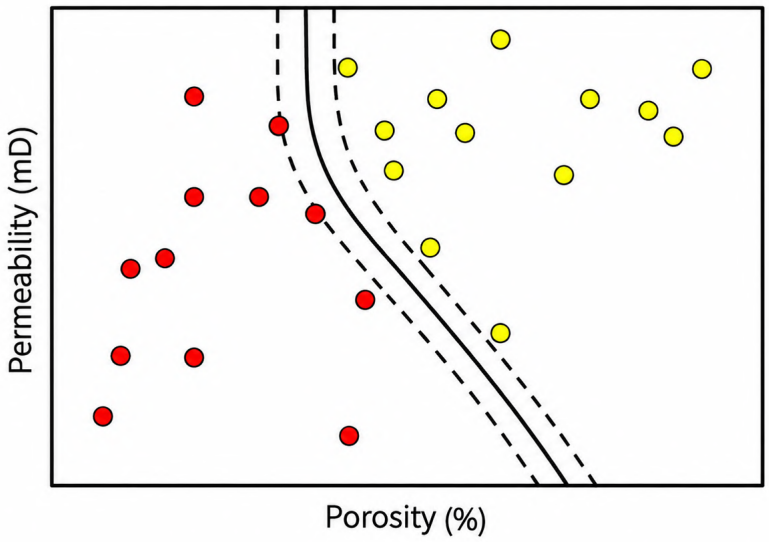
donde:

C (Parámetro de regularización): Controla el equilibrio entre la maximización del margen de separación y la penalización de los errores de clasificación.

ϵ_i (Variables de holgura): Representan el grado en que cada observación viola las restricciones del margen de separación, permitiendo cierta tolerancia a los errores de clasificación.

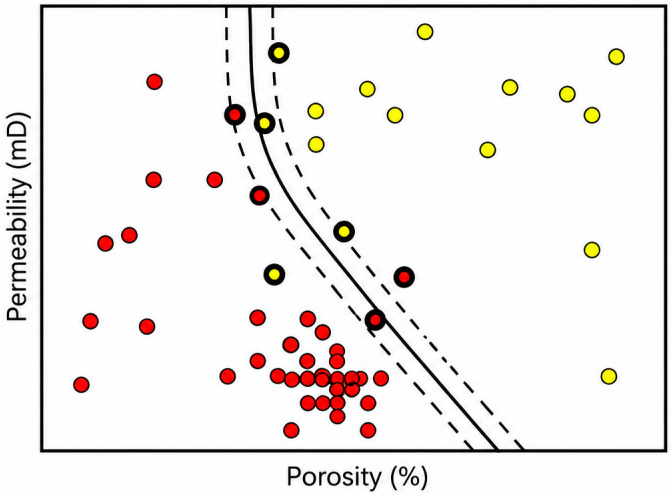
Steps

1. Form a boundary with the largest possible separation between the different cases.



Best separation of the training data of two categories.

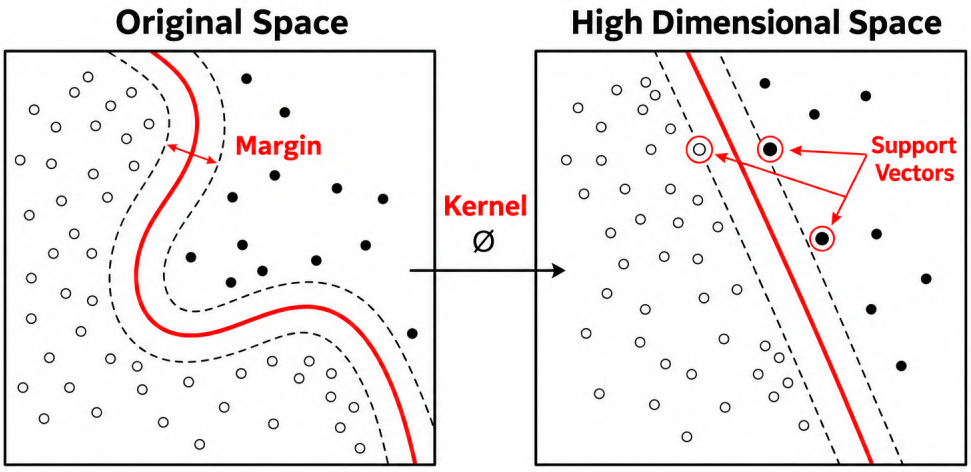
2. Data within the margin or misclassified update the model, they are called support vectors.



Optimization: maximize margin – minimize the regularized error jointly

Training data well within the correct region have no impact on the model!

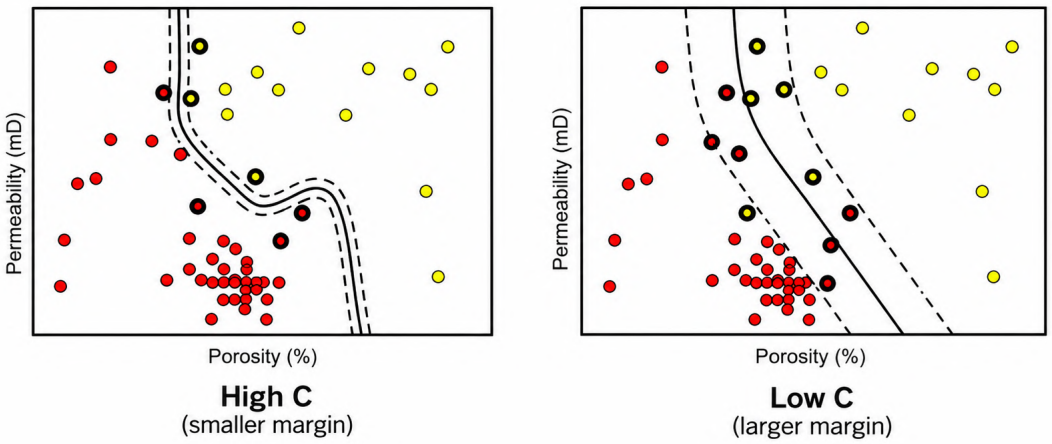
3. Project problem into a higher dimensional space to impose nonlinearity with a variety of kernels.



Linear in the high dimensional space

A very simple model in that space!

4. The parameter controls the penalty of misclassification, high C will result in a more complicated model (lower model bias, higher model variance).



C is a regularization parameter (similar to ridge regression) that controls the fit.

A high C leads to a smaller margin, fewer support vectors, and a higher tendency to overfit.

A low C leads to a larger margin, more support vectors, and a higher tendency to underfit.

1.5 Métodos de aprendizaje no supervisados

Unsupervised Machine Learning (ML) is a category of machine learning in which algorithms analyze datasets without predefined target variables or labels. Unlike supervised learning, where models learn relationships between input features and known outputs, unsupervised learning aims to discover hidden patterns, structures, similarities, and relationships within the data.

The main objective is to identify meaningful information from datasets without explicitly telling the algorithm what it should predict. For example, in petroleum engineering, unsupervised learning can be used to identify reservoir zones with similar petrophysical properties, group wells based on production behavior, or classify geological facies using well-log measurements.

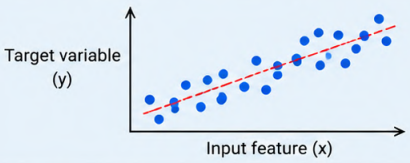
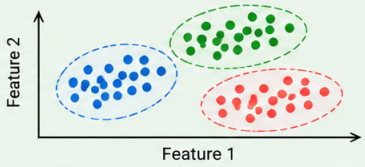
Aspect	Supervised Learning	Unsupervised Learning
Training data	Labeled data (input features with known target outputs)	Unlabeled data (only input features, no target outputs)
Target variable	Required	Not required
Main objective	Learn a mapping from inputs to outputs for prediction or classification	Discover hidden patterns, structures, or relationships in the data
Typical problems	Regression (predict continuous values), Classification (predict categories)	Clustering, dimensionality reduction, anomaly detection
Example algorithms	Linear Regression, Logistic Regression, Support Vector Machine (SVM), Random Forest, Neural Networks	K-Means, Hierarchical Clustering, DBSCAN, Principal Component Analysis (PCA), t-SNE, Autoencoders
Example application	Predict permeability from porosity, classify rock type from well logs	Group reservoir rocks with similar properties, identify well clusters, detect anomalous production
Illustration		
Evaluation metrics	RMSE, MAE, accuracy, precision, recall, F1-score	Silhouette score, inertia (within-cluster sum of squares), cluster interpretability

Figura 8: Ecuación de estado de gases reales

En el aprendizaje no supervisado, únicamente se proporcionan al algoritmo las variables de entrada X; no existe una variable objetivo conocida que indique cuál debería ser la respuesta.

El propósito es que el algoritmo explore los datos e identifique por sí mismo patrones, estructuras, similitudes o representaciones.

$$X \longrightarrow \text{Estructuras o patrones}$$

Algunas de sus principales aplicaciones son:

1. **Clustering**: agrupar observaciones que presentan características similares.
2. **Reducción de dimensionalidad**: representar información de alta dimensión utilizando un menor número de variables.
3. **Detección de anomalías**: identificar observaciones que presentan un comportamiento diferente al resto.
4. **Extracción de características**: obtener nuevas representaciones que concentren información relevante.

Por ejemplo, un algoritmo de clustering puede analizar registros de pozos y encontrar grupos con características similares que posteriormente podrían interpretarse como electrofacies. Algunos métodos comunes son K-Means, clustering jerárquico, DBSCAN, PCA y autoencoders.



Figura 9: Ecuación de estado de gases reales

Idea clave: el algoritmo no conoce previamente la respuesta; busca descubrir la estructura existente en los datos.

1.5.1 K-means

K-Means is one of the most widely used unsupervised learning algorithms. It is a centroid-based clustering method that partitions a dataset into a predefined number of groups, denoted by K. The algorithm aims to group observations such that:

- Observations within the same cluster are as similar as possible.
- Observations in different clusters are relatively dissimilar.
- Each cluster is represented by a centroid, which is the mean position of its assigned observations in feature space.

K-Means is particularly useful when the data contain relatively compact, well-separated groups.

Mathematical formulation

Consider a dataset containing N observations (o) each described by d features:

$$O = \{\mathbf{o}_1, \mathbf{o}_2, \ldots, \mathbf{o}_N\}$$

Each observation is a vector:

$$\mathbf{o}_i = [o_{i1}, o_{i2}, \ldots, o_{id}]^T$$

For example, if we have 5 features ϕ, ρ, S_w, AI,q:

$$\mathbf{o}_i = [\phi_i, \rho_i, S_{wi}, AI_i, q_i]^T$$

The objective is to partition the dataset into K clusters:

$$C_1, C_2, \ldots, C_K$$

- Each cluster is represented by its centroid, denoted by $\boldsymbol{\mu}_k$.

- Objective function: Within-Cluster Sum of Squares (WCSS). K-Means minimizes the sum of squared Euclidean distances between observations and their assigned centroids:

$$J = \sum_{k=1}^{K} \sum_{\mathbf{o}_i \in C_k} \|\mathbf{o}_i - \boldsymbol{\mu}_k\|_2^2$$

Where:
- K: Number of clusters.
- C_k: Set of observations assigned to cluster k.
- $\mathbf{o}_i$: Feature vector of observation i.
- $\boldsymbol{\mu}_k$: Centroid of cluster k.
- $\|\mathbf{o}_i - \boldsymbol{\mu}_k\|_2^2$: Squared Euclidean distance.

For a two-dimensional dataset (ϕ y ρ) the Ssquared Euclidean distance is as follows:

$$d(\mathbf{o}_i, \boldsymbol{\mu}_k) = \sqrt{(\phi_i - \mu_{k\phi})^2 + (\rho_i - \mu_{k\rho})^2}$$

Step 1. Initialize K centroids

The first step consists of selecting the number of clusters K and initializing their centroids.
For example, if $K = 3$, we initialize:

$$\boldsymbol{\mu}_1^{(0)}, \quad \boldsymbol{\mu}_2^{(0)}, \quad \boldsymbol{\mu}_3^{(0)}$$

Common initialization methods include random selection of data points and K-Means++, which selects initial centroids to encourage greater separation.

Step 2. Assign observations to the nearest centroid

For every observation $\mathbf{o}_i$, calculate its distance to all centroids.

The observation is assigned to the cluster whose centroid has the smallest squared Euclidean distance:

$$c_i^{(t)} = \underset{k\in\{1,\dots,K\}}{\arg\min} \|\mathbf{o}_i - \boldsymbol{\mu}_k^{(t)}\|_2^2$$

Where $c_i^{(t)}$ represents the cluster assigned to observation i at iteration t.

For example, consider:

$$\mathbf{o}_i = (2, 3)$$

$$d(\mathbf{o}_i, \boldsymbol{\mu}_k) = \sqrt{(\phi_i - \mu_{k\phi})^2 + (\rho_i - \mu_{k\rho})^2}$$

And two centroids $(k = 2)$:

$$\boldsymbol{\mu}_1 = (1, 2), \quad \boldsymbol{\mu}_2 = (6, 7)$$

The distances are:

$$d_1 = \sqrt{(2-1)^2 + (3-2)^2} = \sqrt{2}$$

$$d_2 = \sqrt{(2-6)^2 + (3-7)^2} = \sqrt{32}$$

Step 3. Update the centroids

After assigning all observations, the centroid of each cluster is recalculated as the arithmetic mean of the observations belonging to that cluster:

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{1}{|C_k^{(t)}|} \sum_{\mathbf{x}_i \in C_k^{(t)}} \mathbf{o}_i$$

Where $|C_k^{(t)}|$ represents the number of observations assigned to cluster k.

For example, suppose Cluster 1 contains:

$$(2, 3), \quad (4, 5), \quad (6, 4)$$

Its updated centroid is:

$$\boldsymbol{\mu}_1 = \left(\frac{2+4+6}{3}, \frac{3+5+4}{3}\right)$$

$$\boldsymbol{\mu}_1 = (4, 4)$$

The centroid therefore moves to the mean position of its assigned observations.

Step 4. Repeat until convergence

Steps 2 and 3 are repeated until the cluster assignments or centroids stabilize. A common convergence criterion is:

$$\max_k \|\boldsymbol{\mu}_k^{(t+1)} - \boldsymbol{\mu}_k^{(t)}\|_2 < \varepsilon$$

Where ε is a predefined tolerance. In practice, implementations may use a different centroid-shift measure or stop when a maximum number of iterations is reached. The resulting clusters represent the final partition identified by the algorithm.

Advantages of K-Means Clustering

#	Advantage	Description
1	Simplicity	Easy to understand, implement, and interpret.
2	Computational efficiency	Fast and requires relatively low computational resources.
3	Scalability	Suitable for large datasets containing thousands or millions of observations.
4	Multidimensional capability	Can analyze datasets with multiple features or dimensions simultaneously.
5	Interpretability	Cluster centroids represent the average characteristics of each group.
6	Convergence	Guaranteed to converge to a local optimum in a finite number of iterations under standard conditions.

Figura 10: Ecuación de estado de gases reales

Disadvantages of K-Means Clustering

#	Disadvantage	Description
1	Requires predefined K	The number of clusters (K) must be specified before running the algorithm.
2	Sensitive to initialization	Different initial centroid positions can lead to different clustering results.
3	Sensitive to outliers	Extreme observations can significantly shift centroids because they are calculated using arithmetic means.
4	Sensitive to feature scaling	Features with larger numerical ranges can dominate distance calculations, leading to misleading clusters.
5	Assumes compact, convex clusters	Performs best with approximately spherical and similarly sized clusters; struggles with irregular or non-convex shapes.
6	Poor performance with varying densities	May incorrectly partition clusters with significantly different densities or sizes.
7	High-dimensional limitations	In high-dimensional spaces, distances can become less informative, and irrelevant features can introduce noise.
8	Limited to numerical features	Standard K-Means relies on Euclidean distances and arithmetic means, making it unsuitable for purely categorical variables without modification.

Figura 11: Ecuación de estado de gases reales

Optimal number of clusters

The Elbow Method is a graphical technique used to estimate the optimal number of clusters (K) in K-Means clustering. One of the main limitations of K-Means is that the number of clusters must be specified before training. However, the appropriate number of clusters is often unknown.

The Elbow Method addresses this problem by evaluating how the Within-Cluster Sum of Squares (WCSS) changes as the number of clusters increases.

The fundamental idea is to identify a value of K beyond which adding more clusters produces only a relatively small improvement in clustering compactness.

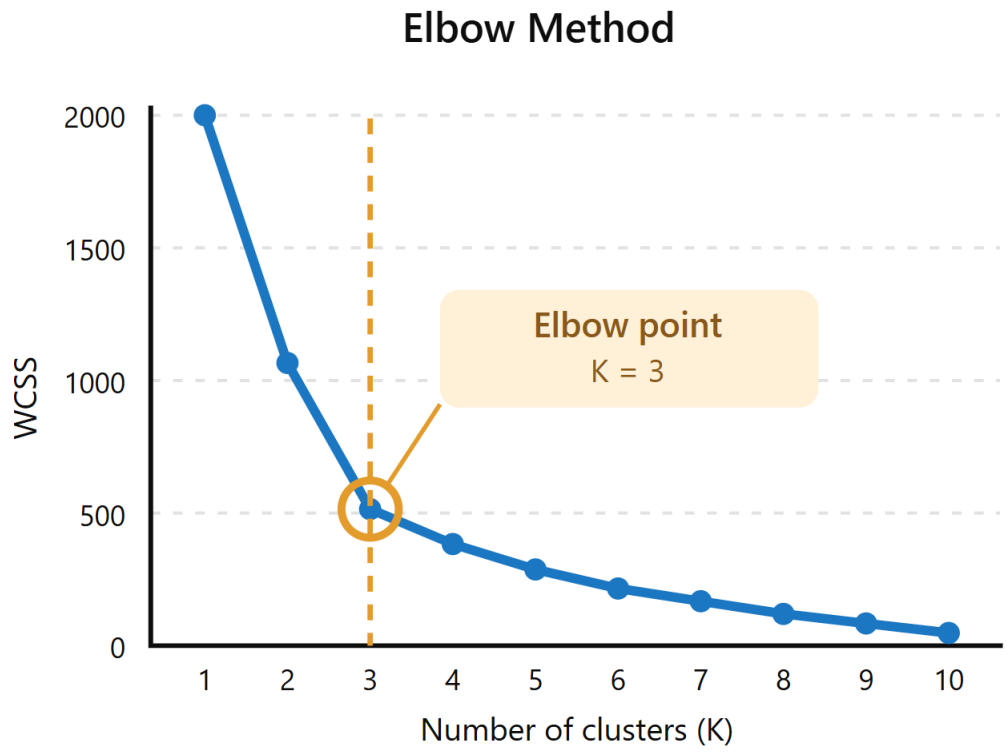


Figura 12: Ecuación de estado de gases reales

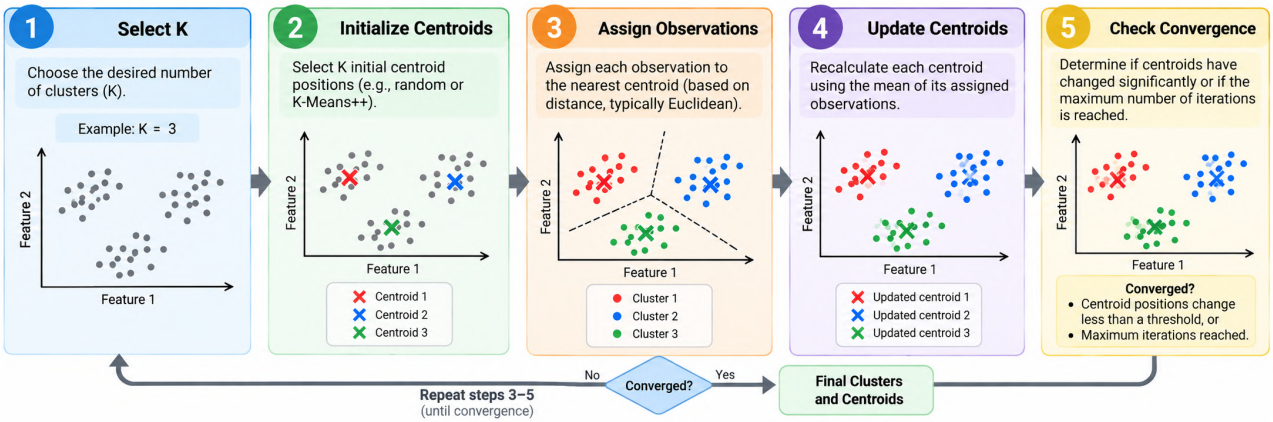


Figura 13: Ecuación de estado de gases reales

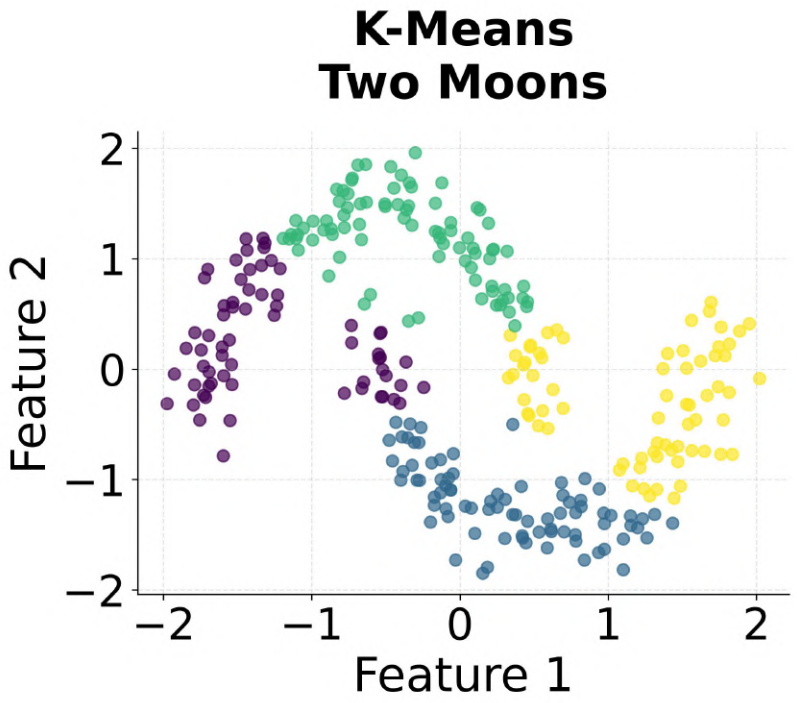


Figura 14: Ecuación de estado de gases reales

1.5.2 Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

DBSCAN is an unsupervised machine learning algorithm that identifies clusters based on the density of observations within a dataset. Unlike K-Means, which groups observations according to their distance from predefined centroids, DBSCAN identifies regions containing a high concentration of data points and separates them from regions with low point density.

The algorithm was introduced by Ester et al. (1996) and is particularly useful for identifying clusters with irregular shapes and detecting outliers. Main characteristics

- Density-based clustering: Groups observations located in densely populated regions.
- No predefined number of clusters: The algorithm automatically determines the number of clusters from its parameters and the dataset.
- Outlier detection: Identifies observations that do not belong to sufficiently dense regions.
- Arbitrary cluster shapes: Can detect non-spherical and irregularly shaped clusters.
- Multidimensional capability: Can operate on datasets containing multiple numerical features.

How DBSCAN works
DBSCAN uses two main hyperparameters to determine whether observations belong to dense regions.

1. Epsilon (ε or eps): Maximum neighborhood radius around an observation.
2. MinPts (min_samples): Minimum number of observations required within the epsilon neighborhood for a point to be considered a core point, including the point itself.

For an observation $\mathbf{x}_i$, its epsilon neighborhood is defined as:

$$N_\varepsilon(\mathbf{x}_i) = \{\mathbf{x}_j \in X : d(\mathbf{x}_i, \mathbf{x}_j) \leq \varepsilon\}$$

Where:
- $N_\varepsilon(\mathbf{x}_i)$: Set of observations within the epsilon radius.
- $d(\mathbf{x}_i, \mathbf{x}_j)$: Distance between observations i and j.
- ε: Neighborhood radius.
For numerical features, Euclidean distance is commonly used:

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{m=1}^{D} (x_{im} - x_{jm})^2}$$

Here, D is the number of features or dimensions.

Types of points in DBSCAN
DBSCAN classifies observations into three categories:

A. Core points
A core point has at least MinPts observations within its epsilon neighborhood, counting itself.

$$|N_\varepsilon(\mathbf{x}_i)| \geq MinPts$$

Core points are located in dense regions and are responsible for expanding clusters.

B. Border points
A border point has fewer than MinPts observations in its own neighborhood but lies within the epsilon neighborhood of at least one core point.

$$|N_\varepsilon(\mathbf{x}_i)| < MinPts$$

Border points can belong to a cluster but cannot independently expand it.

C. Noise points (outliers)
Noise points are neither core points nor border points. They do not belong to any cluster identified by DBSCAN. In Scikit-learn, noise observations receive the label -1.

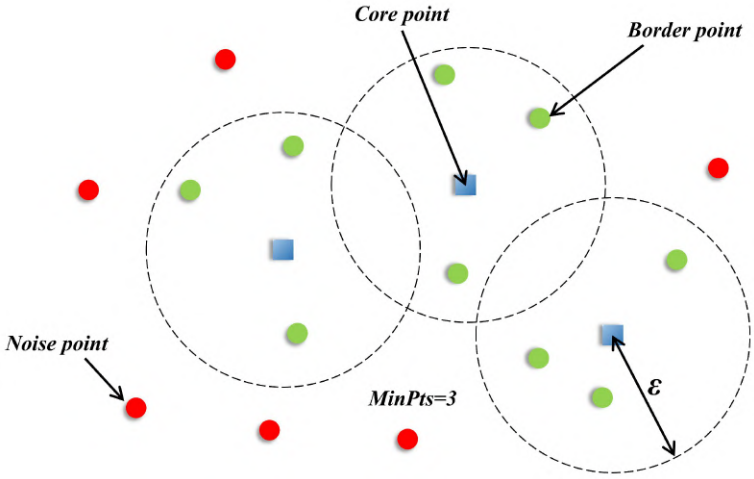


Figura 15: Ecuación de estado de gases reales

DBSCAN algorithm: Step-by-step

- Step 1 — Define hyperparameters. Select ε and MinPts.
- Step 2 — Select an unvisited observation. Calculate the distances between this observation and its neighboring observations.
- Step 3 — Identify core points. If the observation has at least MinPts points within its epsilon neighborhood, it becomes a core point and starts or expands a cluster.
- Step 4 — Expand the cluster. All density-reachable observations are included in the cluster. Expansion continues through other core points; border points are included but do not expand the cluster.
- Step 5 — Repeat. Continue processing observations until all have been visited or assigned. Observations that are neither core nor border points are labeled as noise.
- Unlike K-Means, DBSCAN does not iteratively update centroids.

Selecting DBSCAN hyperparameters

A. Selecting MinPts
There is no universally optimal value of MinPts.
A common heuristic is:

$$MinPts \geq D + 1$$

Where D is the number of dimensions.
For example, a dataset with five features might start with MinPts = 6, although larger values may be more robust to noise.

B. Selecting epsilon using the k-distance plot
One commonly used technique is the k-nearest-neighbor distance plot.
For each observation, calculate its distance to the $k-th$ nearest neighbor, including the observation itself when using the Scikit-learn convention. Sort these distances and plot them.



Figura 16: Ecuación de estado de gases reales

Optimal number of clusters

Unlike K-Means, which requires specifying the number of clusters (K), DBSCAN automatically identifies clusters based on the density of observations.

However, DBSCAN requires two important hyperparameters:

- Epsilon (ε or eps): The maximum distance between two observations for them to be considered neighbors.
- MinPts (min_samples): The minimum number of observations required within an epsilon neighborhood for a point to be classified as a core point, including the point itself in Scikit-learn.

One of the main challenges when applying DBSCAN is selecting an appropriate value of ε. The K-Distance Elbow Method, also called the k-nearest-neighbor distance plot, is a graphical technique used to estimate a suitable epsilon value. Importantly, the Elbow Method for DBSCAN is different from the one used for K-Means.

The procedure consists of five steps:
1. Select a value for min_samples, for example, 5.
2. Calculate the distance from each observation to its 5th nearest neighbor, counting the observation itself.
3. Sort these distances in ascending order.
4. Plot the sorted distances against the observation index.
5. Identify the elbow or knee, where distances begin to increase rapidly.

The distance at this transition provides a candidate value for ε.

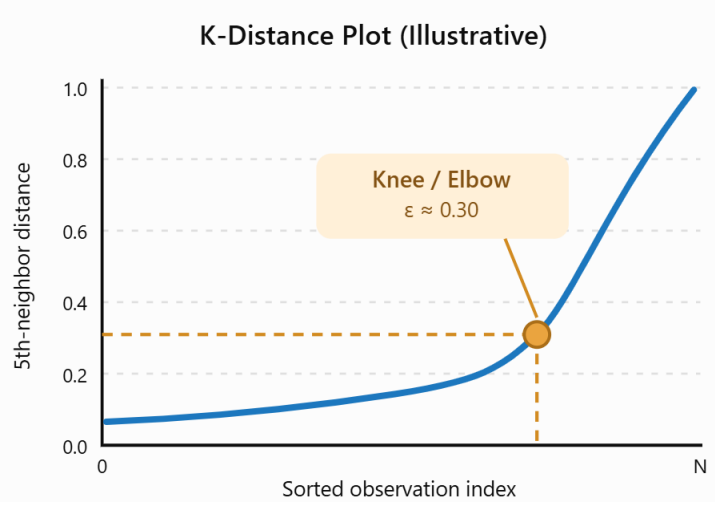


Figura 17: Ecuación de estado de gases reales

1.5.3 Spectral Clustering

Spectral Clustering is an unsupervised machine learning algorithm that groups observations based on their similarities rather than directly minimizing distances to cluster centroids, as K-Means does. The central idea is to represent a dataset as a graph, where observations are nodes and connections between them indicate similarity. Spectral Clustering then uses the eigenvalues and eigenvectors of a matrix derived from this graph to identify groups of strongly connected observations. Unlike K-Means, which generally works best with compact, approximately spherical clusters, Spectral Clustering can identify non-convex, irregular, and interconnected geometric structures.

Why is it called "Spectral"?

The "magic" of how it finds the best place to cut the network comes from Linear Algebra. It analyzes the spectrum (the set of eigenvalues and eigenvectors) of a special matrix derived from the graph, called the Laplacian Matrix. The eigenvectors of this matrix have an amazing property: they reveal the underlying connectivity and structure of the graph. This process essentially "unrolls" the complex shapes into a simpler, lower-dimensional space where the clusters are easy to separate.

Step 1. Construct a similarity graph

Consider a dataset containing n observations, each described by p features:

$$X = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$$

Each observation becomes a node in a graph. Two observations are connected if they are sufficiently similar. Common approaches include:

- K-nearest neighbors (KNN): Connect each observation to its nearest neighbors.
- Radius-based neighbors: Connect observations separated by less than a specified distance.
- Fully connected graph: Connect all observations, with weights determined by similarity.

Step 2. Calculate the similarity matrix

The similarity matrix, also called the affinity matrix, describes the strength of the connections:

$$W = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n1} & w_{n2} & \cdots & w_{nn} \end{bmatrix}$$

One commonly used similarity function is the Gaussian radial basis function (RBF):

$$w_{ij} = \exp\left(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\right)$$

Where:

- w_{ij}: Similarity between observations i and j.
- $\|\mathbf{x}_i - \mathbf{x}_j\|^2$: Squared Euclidean distance.
- γ: Parameter controlling how quickly similarity decreases with distance.

Two nearby observations have similarity close to 1, while distant observations have similarity approaching 0.

For example, using $\gamma = 0.5$:

Squared distance	*Similarity*
0	1
1	0.607
4	0.135
9	0.011

Step 3. Construct the graph Laplacian

First, calculate the degree matrix D, a diagonal matrix containing the sum of each row of the similarity matrix:

$$D_{ii} = \sum_{j=1}^{n} w_{ij}$$

The unnormalized graph Laplacian is:

$$L = D - W$$

A commonly used normalized version is:

$$L_{\text{sym}} = I - D^{-1/2} W D^{-1/2}$$

Where I is the identity matrix.

The graph Laplacian captures how observations are connected. Its eigenvectors contain information about the graph's underlying structure and can help separate groups with strong internal connections and weak connections between groups.

Step 4. Calculate eigenvalues and eigenvectors

For the normalized Laplacian, solve:

$$L_{\text{sym}}\mathbf{v} = \lambda\mathbf{v}$$

Where:

- λ: Eigenvalue.
- $\mathbf{v}$: Corresponding eigenvector.

For K clusters, a common spectral clustering formulation selects the eigenvectors corresponding to the \(K\) smallest eigenvalues, then normalizes their rows to construct a new representation of the observations.

$$U = [\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_K]$$

This new representation is called the spectral embedding.

An important concept is that the number of zero eigenvalues of the graph Laplacian equals the number of connected components in the graph. In real datasets, small eigenvalues can provide information about weakly connected groups.

Step 5. Apply K-Means to the spectral embedding

Once the observations have been transformed into the spectral feature space, a clustering algorithm is applied. For example:

$$U \longrightarrow \text{K-Means} \longrightarrow K \text{ clusters}$$

The critical difference is that K-Means is no longer applied directly to the original variables, but to the transformed representation derived from graph connectivity.

This makes it possible to identify clusters that are not easily separated in the original feature space.

Numerical example: Similarity and graph Laplacian

Consider three observations with two features:

$observation$	$Feature1$	$Feature2$
A	1	1
B	2	1
C	5	5

Using the RBF similarity function with $\gamma = 0.5$:

$$w_{AB} = e^{-0.5[(1-2)^2+(1-1)^2]} = 0.6065$$

$$w_{AC} = e^{-0.5[(1-5)^2+(1-5)^2]} \approx 0.00000011$$

$$w_{BC} = e^{-0.5[(2-5)^2+(1-5)^2]} \approx 0.00000373$$

For a graph without self-loops, set $w_{ii} = 0$. The resulting similarity matrix is approximately:

$$W = \begin{bmatrix} 0 & 0.6065 & 0.0000 \\ 0.6065 & 0 & 0.000004 \\ 0.0000 & 0.000004 & 0 \end{bmatrix}$$

The degree matrix is:

$$D \approx \begin{bmatrix} 0.6065 & 0 & 0 \\ 0 & 0.6065 & 0 \\ 0 & 0 & 0.000004 \end{bmatrix}$$

The unnormalized Laplacian is $L = D - W$.

In this example, observations A and B have a strong connection, while C is only weakly connected to either of them. This suggests a natural grouping of A and B in one cluster and C in another.

This example illustrates the similarity-graph construction; the full spectral algorithm additionally uses the Laplacian eigenvectors to assign cluster labels.

Spectral Clustering can be useful in reservoir characterization when the goal is to identify groups of observations with complex relationships.

Examples include:

- Electrofacies classification: Group well-log measurements using gamma ray, density, neutron porosity, resistivity, and sonic logs.
- Rock typing: Identify groups with similar porosity, permeability, and pore-system characteristics.
- Reservoir zonation: Cluster grid cells based on petrophysical attributes and spatial connectivity.
- Production behavior: Identify wells with similar production profiles or decline characteristics.
- Fractured reservoirs: Investigate connected regions with complex geometric structures.

For spatial reservoir analysis, similarity can incorporate both petrophysical and spatial information. However, features must be scaled or weighted carefully so that geographic distance does not overwhelm geological properties.

Characteristic	K-Means	Spectral Clustering
Main principle	Distance to centroids	Similarity graph
Cluster geometry	Compact, convex	Can be non-convex
Centroids	Yes	Not required
Main computation	Iterative centroid updates	Eigenvector computation
Large datasets	Generally efficient	Can be expensive
Main hyperparameters	K, initialization	K, affinity, graph connectivity
Sensitivity	Scaling and initialization	Scaling and graph construction

Figura 18: Ecuación de estado de gases reales

1.6 Métodos de aprendizaje semisupervisado

El aprendizaje semisupervisado se encuentra entre los dos enfoques anteriores. Utiliza una pequeña cantidad de datos etiquetados junto con una cantidad considerablemente mayor de datos sin etiquetar.

$$\textit{Datos etiquetados} + \textit{Datos sin etiquetar} \longrightarrow \textit{Modelo}$$

Su principal ventaja aparece cuando obtener observaciones es relativamente sencillo, pero determinar la respuesta correcta para cada observación es costoso o requiere especialistas.

Por ejemplo, podemos disponer de miles de muestras de datos sísmicos, imágenes o registros geofísicos, pero solamente una pequeña parte puede haber sido interpretada y clasificada por un geocientífico.

El modelo aprovecha la información de las observaciones etiquetadas y la estructura presente en los datos no etiquetados para mejorar su aprendizaje.

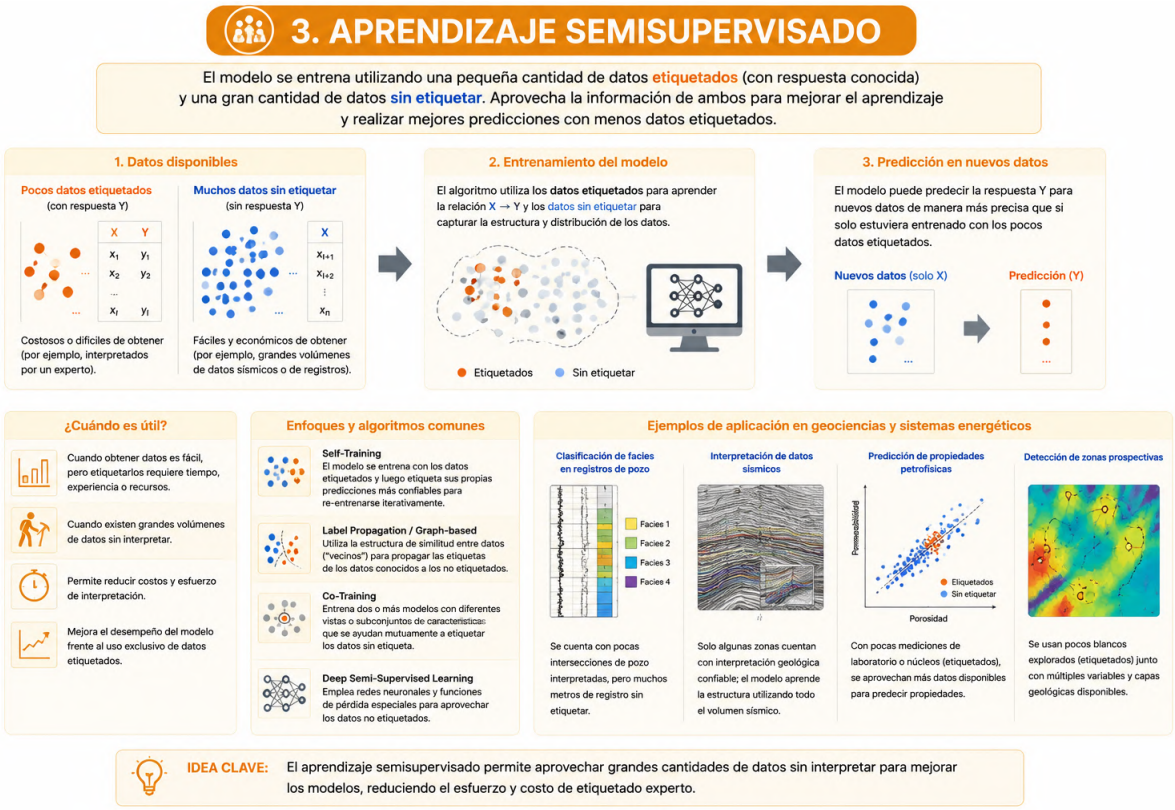


Figura 19: Ecuación de estado de gases reales

Idea clave: utiliza pocas observaciones con respuesta conocida y muchas sin respuesta conocida.

1.7 Métodos de aprendizaje por refuerzo

El aprendizaje por refuerzo funciona de manera diferente. En lugar de aprender directamente de pares X,Y, un agente interactúa con un entorno, toma decisiones y observa las consecuencias de sus acciones.

El proceso puede representarse como:

$$Estado \rightarrow Acción \rightarrow Recompensa$$

El agente recibe una recompensa o penalización dependiendo del resultado de sus decisiones y aprende progresivamente una estrategia o política que busca maximizar la recompensa acumulada.

Por ejemplo, en ingeniería de yacimientos, un agente podría modificar las tasas de producción o inyección y recibir una recompensa relacionada con la producción acumulada, recuperación de hidrocarburos o desempeño económico. Después de múltiples interacciones, buscaría encontrar una estrategia de operación óptima.

Las redes neuronales pueden utilizarse dentro del aprendizaje por refuerzo, especialmente en Deep Reinforcement Learning, pero una red neuronal no es por sí misma un método de aprendizaje por refuerzo.

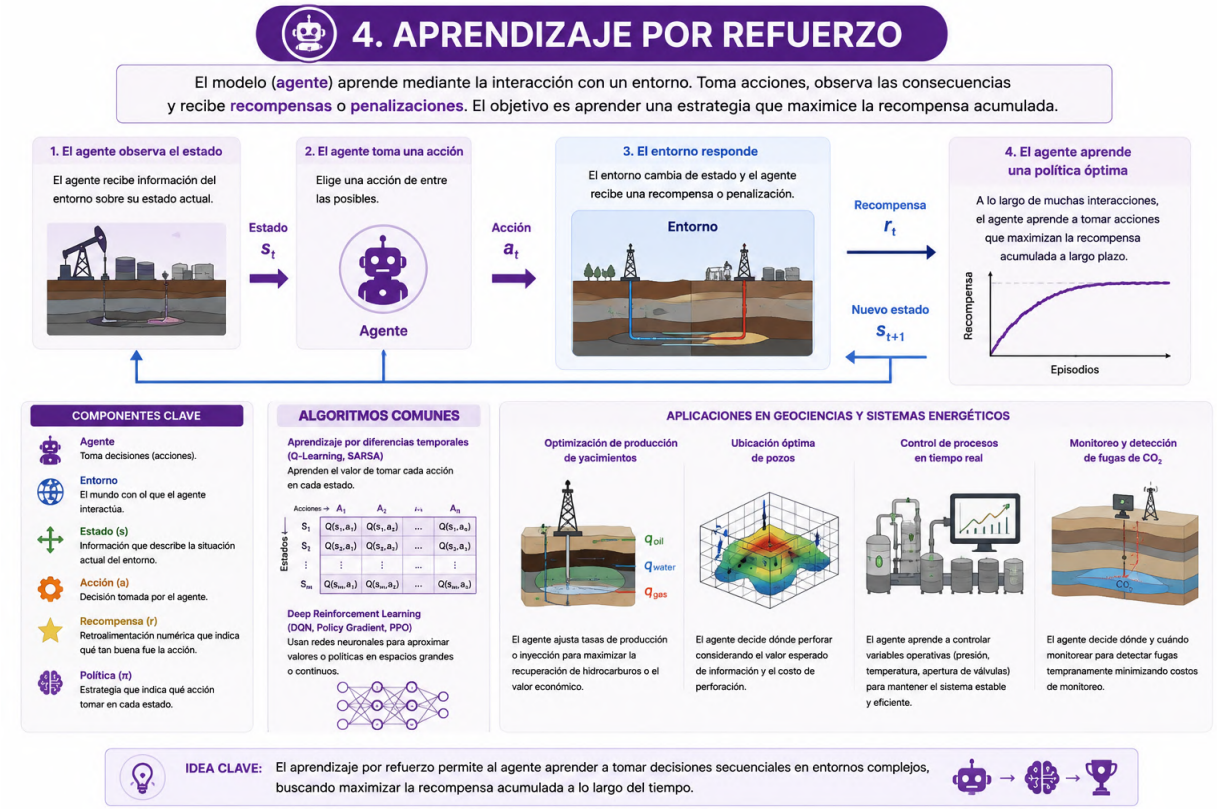


Figura 20: Ecuación de estado de gases reales

Idea clave: el algoritmo aprende principalmente mediante las consecuencias de sus decisiones e interacción con un entorno.